东北大学 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY



Programming in Python
—— PART I Basics

1

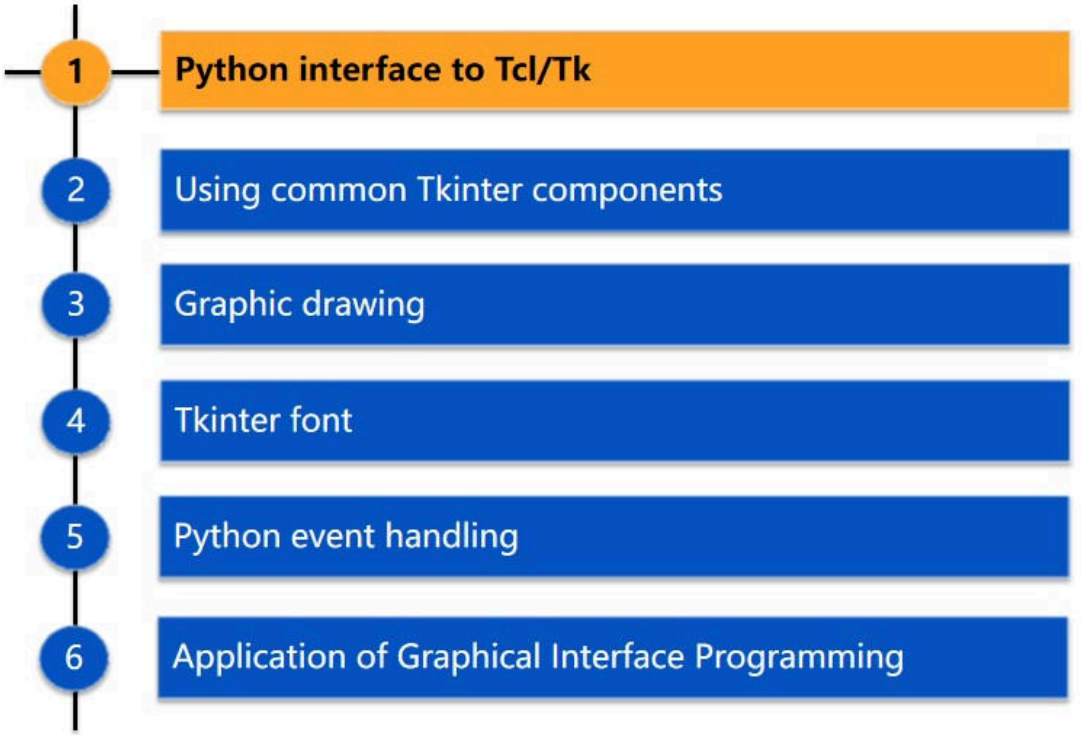
雨课堂 Rain Classroom

Example: Interface implemented by tkinter.pack



Chapter 7: Graphical User Interfaces with Tk

Contents

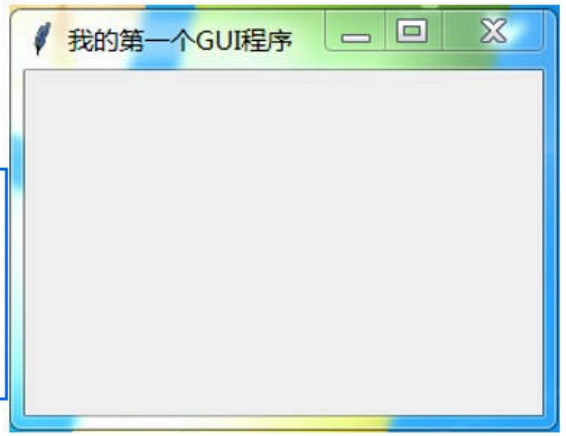


Python interface to Tcl/Tk

A Hello World Program

- Tkinter is Python's standard GUI library, which is built into the Python installation package, so after installing Python, you can use this GUI graphics development library as long as you import Tkinter.
- Python's IDEL is also written in Tkinter.
- Example:

```
from tkinter import *          # is the standard binding to Tk
root = Tk()                    # sets up the main application window
root.title('我的第一个GUI程序')  # set the title
root.mainloop()                # show window
```



Python interface to Tcl/Tk

A Hello World Program

- Example: Create and display a window with an initial size of 800*600

```
from tkinter import *      # 不建议使用这种通配符的导入方式
root = Tk()
root.title('主窗口')
root.geometry("800x600")          # 使用geometry()方法设置窗口的大小
root.minsize(400, 600)            # 设置窗口最小尺寸
root.maxsize(1440, 800)           # 设置窗口最大尺寸
toplevel = Toplevel(root)         # 创建子窗口
toplevel.title('子窗口')
win.mainloop()                    # 进入消息循环，也就是
```

建议这种方式导入

```
import tkinter as tk
root = tk.Tk()
root.title('我的第一个GUI程序')
root.geometry("800x600")
root.minsize(400, 600)
root.maxsize(1440, 800)
……
```

Geometry Manager(几何布局管理器)

- The Tkinter geometry manager is used to organize and manage how child components are laid out within a parent component (often a window).

- Tkinter provides 3 different styles of geometric management classes:
  - pack: geometry manager that packs around edges of cavity(空腔)
  - grid
  - place

- Note: These three geometry management must not be mixed in the same master window!

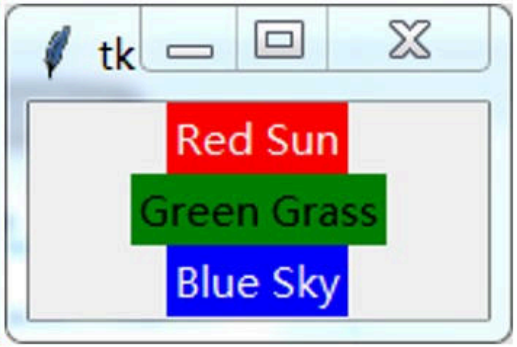
雨课堂 Rain Classroom

Geometry Manager

1. The *Pack* Geometry Manager

- block method, in generation order, fast, common
- syntax: component_object.pack (<option>)

```
import tkinter as tk
root = tk.Tk()
tk.Label(root, text="Red Sun", bg="red", fg="white").pack()
tk.Label(root, text="Green Grass", bg="green", fg="black").pack()
tk.Label(root, text="Blue Sky", bg="blue", fg="white").pack()
root.mainloop()
```



Geometry Manager

1. The *Pack* Geometry Manager

- The parameter options of the Pack method are as follows:

options	optional items	Description
side	top(default), bottom, left, right	Specifies which side of the container the content will be packed against(组件停靠边)
anchor	n, s, e, w, nw, sw, se, ne, center(default)	specifies where to position each content in its parcel(锚点位置)
fill	'x', 'y', 'both', none	how to stretch the content.
expand	boolean——0, 1	Specifies whether the content should be expanded to consume extra space in their container.
ipadx, ipady	Unit is c(cm), m(mm), i(inch), p(printer's)	specifies how much horizontal/vertical internal padding to leave on each side of the content(组件内空隙)
padx, pady	Unit is c(cm), m(mm), i(inch), p(printer's)	specifies how much horizontal/vertical external padding to leave on each side of the content(组件外空隙)

雨课堂 Rain Classroom

Geometry Manager

1. The *Pack* Geometry Manager

- fill:

options	optional items	Description
fill	'x', 'y', 'both', none	how to stretch the content.

```
import tkinter as tk
root = tk.Tk()
w = tk.Label(root, text="Red Sun", bg="red", fg="white")
w.pack(fill = 'x')
w = tk.Label(root, text="Green Grass", bg="green", fg="black")
w.pack(fill = 'x')
w = tk.Label(root, text="Blue Sky", bg="blue", fg="white")
w.pack(fill = 'x')
root.mainloop()
```



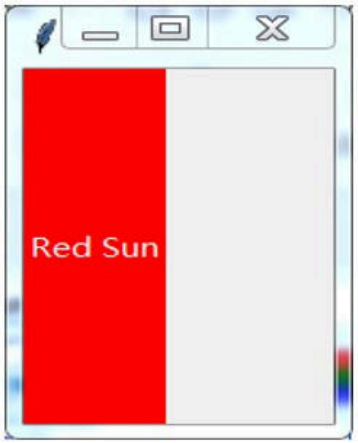
Geometry Manager

1. The *Pack* Geometry Manager

- fill:

options	optional items	Description
fill	'x', 'y', 'both', none	how to stretch the content.

```
import tkinter as tk
root = tk.Tk()
w = tk.Label(root, text="Red Sun", bg="red", fg="white")
w.pack(side = 'left', fill='y')
root.mainloop()
```



Geometry Manager

1. The *Pack* Geometry Manager

- **padx** - specifies how much horizontal external padding to leave on each side of the content.

```
import tkinter as tk
root = tk.Tk()
L = tk.Label(root, text="Red Sun", bg="red", fg="white")
L.pack(fill='x', padx=10)
L = tk.Label(root, text="Green Grass", bg="green", fg="black")
L.pack(fill='x', padx=[10, 20])
L = tk.Label(root, text="Blue Sky", bg="blue", fg="white")
L.pack(fill='x', padx=10)
root.mainloop()
```

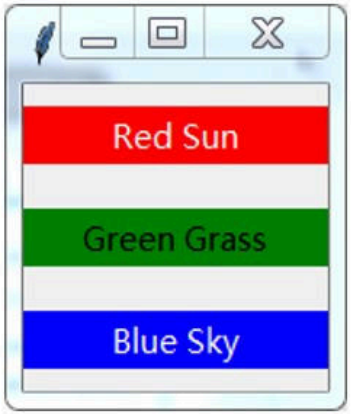


Geometry Manager

1. The *Pack* Geometry Manager

- **pady** - specifies how much vertical external padding to leave on each side of the content.

```
import tkinter as tk
root = tk.Tk()
w = tk.Label(root, text="Red Sun", bg="red", fg="white")
w.pack(fill=X, pady=10)
w = tk.Label(root, text="Green Grass", bg="green", fg="black")
w.pack(fill=X, pady=10)
w = tk.Label(root, text="Blue Sky", bg="blue", fg="white")
w.pack(fill=X, pady=10)
root.mainloop()
```

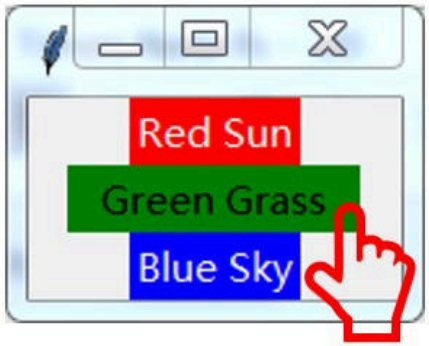


Geometry Manager

1. The *Pack* Geometry Manager

- **ipadx** - specifies how much horizontal internal padding to leave on each side of the content.
  - must be a valid screen distance, such as 2 or .5c. It defaults to 0.

```
import tkinter as tk
root = tk.Tk()
w = tk.Label(root, text="Red Sun", bg="red", fg="white")
w.pack()
w = tk.Label(root, text="Green Grass", bg="green", fg="black")
w.pack(ipadx = 10)
w = tk.Label(root, text="Blue Sky", bg="blue", fg="white")
w.pack()
root.mainloop()
```

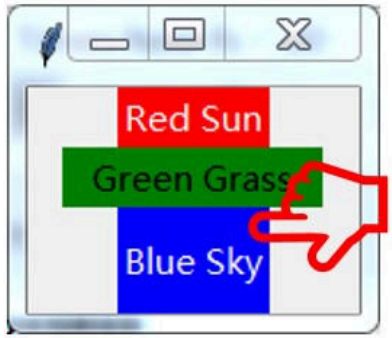


Geometry Manager

1. The *Pack* Geometry Manager

- **ipady** - specifies how much vertical internal padding to leave on each side of the content.

```
import tkinter as tk
root = tk.Tk()
w = tk.Label(root, text="Red Sun", bg="red", fg="white")
w.pack()
w = tk.Label(root, text="Green Grass", bg="green", fg="black")
w.pack(ipadx=10)
w = tk.Label(root, text="Blue Sky", bg="blue", fg="white")
w.pack(ipady=10)
root.mainloop()
```

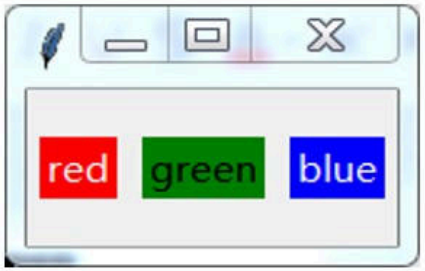


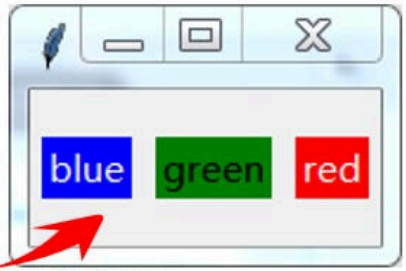
Geometry Manager

1. The *Pack* Geometry Manager

- **side** - Specifies which side of the container the content will be packed against. Defaults to top

```
import tkinter as tk
root = tk.Tk()
w = tk.Label(root, text="red", bg="red", fg="white")
w.pack(padx=5, pady=10, side='left')
w = tk.Label(root, text="green", bg="green", fg="black")
w.pack(padx=5, pady=20, side='left')
w = tk.Label(root, text="blue", bg="blue", fg="white")
w.pack(padx=5, pady=20, side='left')
root.mainloop()
```



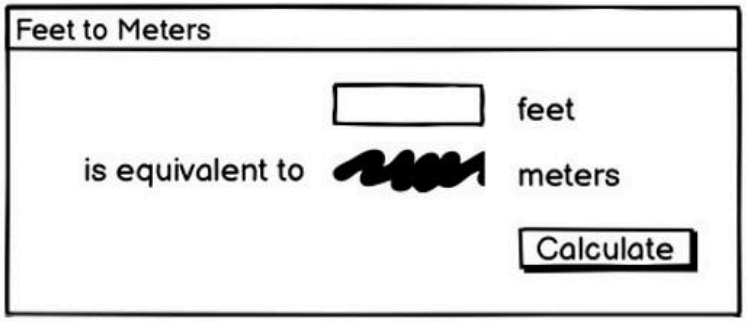


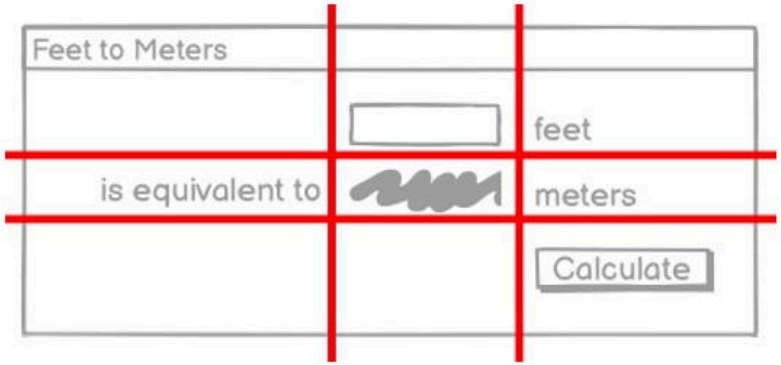
- 如果把上述 side 属性的值都改为 RIGHT, 那么上面 Label 控件的排列顺序就反过来了

Geometry Manager

2. The *Grid* Geometry Manager

- The layout of our user interface, which follows a 3 x 3 grid.



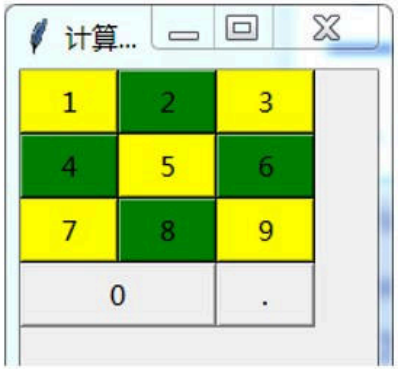


- Grid 在很多场景下是最好用的布局方式。相比而言，Pack 布局在控制细节方面有些力不从心。
- Grid 把控件位置作为一个二维表结构来维护，即按照行列的方式排列控件。控件位置由其所在的行号和列号决定。使用 Grid 布局的过程就是为各个控件指定行号和列号的过程。不需要为每个格子指定大小，Grid 布局会自动设置一个合适的大小。每一列中，列宽由这列最宽的单元格确定；子组件可以跨越多行/列

Geometry Manager

2. The *Grid* Geometry Manager

- Example: calculator




```
import tkinter as tk
root = tk.Tk()
root.geometry('200x200+280+280')
root.title('计算器示例')
color = ('green', 'yellow')              # 定义背景色
for Bi in range(1, 10):
    Li = tk.Button(root, text=str(Bi), width=5, bg=color[Bi%2])
    Li.grid(row=(Bi-1)//3, column=(Bi-1)%3)          # Grid 网格布局
L0 = tk.Button(root, text='0')
L0.grid(row = 3, column = 0, columnspan=2, sticky=E+W )# 跨两列
Lp = tk.Button(root, text='.')
Lp.grid(row = 3, column = 2, sticky=E+W )          # 左右贴紧
root.mainloop()
```

200x200代表了初始化时主窗口的大小，
280, 280代表了初始化时窗口所在的位置

Geometry Manager

2. The *Grid* Geometry Manager

- **syntax: component_object.grid (option)**
- The parameter options of the Grid method are as follows:

options	optional items	Description
sticky	n, s, e, w, nw, sw, se, ne, center(default)	If a content's cell is larger than its requested dimensions, this option may be used to position (or stretch) the content within its cell.(组件在单元格内的位置或拉伸)
row, column	0-n， integer	The component occupies the nth row/column in the grid.(组件放置的单元格位置，即行列数)
rowspan, columnspan	default is one， integer	The component occupies n rows/columns in the grid.(行/列跨度)
ipadx, ipady	Unit is c(cm), m(mm), i(inch), p(printer's)	specifies how much horizontal/vertical internal padding to leave on each side of the content(组件内空隙)
padx, pady	Unit is c(cm), m(mm), i(inch), p(printer's)	specifies how much horizontal/vertical external padding to leave on each side of the content(组件外空隙)

雨课堂 Rain Classroom

Geometry Manager

3. The *Place* Geometry Manager

- The placer is a geometry manager for Tk.
- It provides simple fixed placement of windows, where you **specify the exact size and location** of one window, called the content, within another window, called the container.
- The placer also provides **rubber-sheet placement**, where you specify the size and location of the content in terms of(按照) the dimensions(大小) of the container, so that the **content changes size and location** in response to changes in the size of the container.
- Lastly, the placer allows you to mix these styles of placement so that, for example, the content has a fixed width and height but is centered inside the container.
- Syntax: **component_object.place (<option>)**

雨课堂 Rain Classroom

Geometry Manager

3. The *Place* Geometry Manager

- The parameter options of the Place method are as follows:

options	optional items	Description
anchor	n, s, e, w, nw(default) , sw, se, ne, center	specifies which point of window is to be positioned at the (x,y) location selected by the -x, -y, -relx, and -rely options.
bordermode	inside, outside	Mode determines the degree to which borders within the container are used in determining the placement of the content.(决定组件在容器内位置是否考虑边界)
x, y	integer	specifies the x/y -coordinate within the container window of the anchor point for window.(组件位置绝对像素)
relx, rely	0.0 - 1.0	specifies the x/y -coordinate within the container window of the anchor point for window(组件相对位置，0.0左边缘，1.0右边缘，0.5中心)
height, width	0-n， integer	specifies the height/width for window in screen units.(组件高度和宽度)

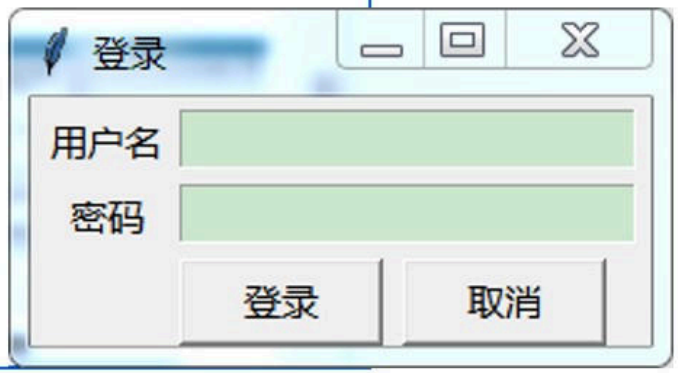
雨课堂 Rain Classroom

Geometry Manager

3. The *Place* Geometry Manager

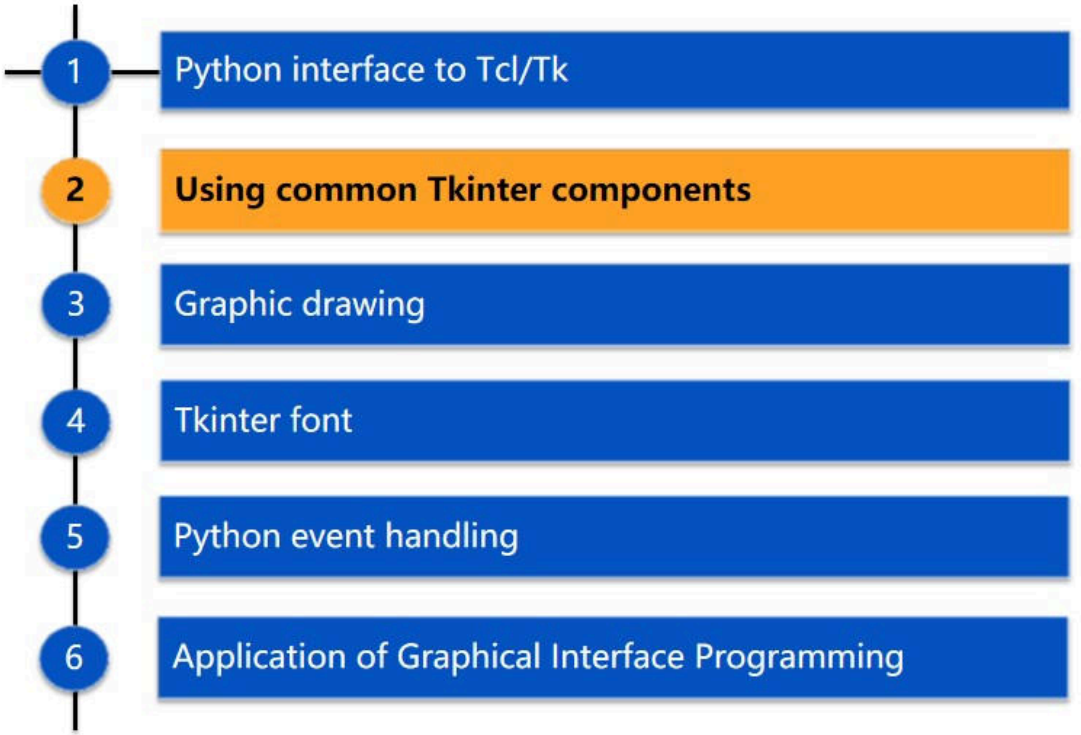
- Example:

```
import tkinter as tk
root = tk.Tk()
root.title("登录")
root['width']=250; root['height']=100                    # root.geometry('250x100')
tk.Label(root, text = '用户名', width=6).place(x=1, y=5)
tk.Entry(root,width=20).place(x=60,y=5)
tk.Label(root,text = '密码',width=6).place(x=1,y=35)
tk.Entry(root,width=20, show='*').place(x=60,y=35)
tk.Button(root,text = '登录',width=8).place(x=60,y=65)
tk.Button(root,text = '取消',width=8).place(x=150,y=65)
root.mainloop()
```



Chapter 7: Graphical User Interfaces with Tk

Contents



雨课堂 Rain Classroom

Basic Widgets

- This section introduces the basic Tk widgets that you'll find in just about any user interface: frames, labels, buttons, checkbuttons, radiobuttons, entries, and comboboxes.

Name	Description
Frame	Frames often act as master widgets for a geometry manager like grid, which manages the slave widgets contained within the frame. (作为容器管理其他组件)
Label	A label is a widget that displays text or images
Button	A button, unlike a frame or label, is very much there to interact with. Users press a button to perform an action.
Checkbutton	Checkbutton widgets are frequently used to allow users to turn an option on or off.(多选)
Radiobutton	A radiobutton widget lets you choose between one of several mutually exclusive choices.(单选)
Entry	An entry widget presents users with a single-line text field where they can type in a string value.(单行输入框)
Combobox	A combobox widget combines an entry with a list of choices. (组合框，即可下拉也可输入)

雨课堂 Rain Classroom

Basic Widgets

- A object instance of the component class can be created through its constructor(构造函数) , such as:

```
import tkinter as tk
root = tk.Tk()
button1 = tk.Button(root, text = "确定").pack(side = RIGHT)
button2 = tk.Button(root, text = "取消").pack(side = RIGHT)
root.mainloop()
```




```
import tkinter as tk        # is the standard binding to Tk
from tkinter import ttk     # is a submodule of tkinter, a newer "themed widgets"
                            # that were added to Tk in 8.5.
root = tk.Tk()
button1 = ttk.Button(root, text = "确定").pack(side = RIGHT)
button2 = trk.Button(root, text = "取消").pack(side = RIGHT)
root.mainloop()
```

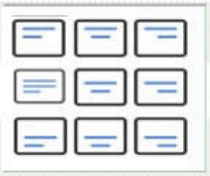


Basic Widgets

standard options (标准属性)

- All components will have some standard options like size, font and color etc. Commonly used standard options are shown in the table.

options	Description
dimension	component size
color	component color
font	component font
anchor	n, s, e, w, nw, sw, se, ne, center(default)
relief	component style (控件样式)
bitmap	build-in bitmap： error, gray75, gray50, gray25, gray12, hourglass, info, questhead, question, warning.
text	Displaying Text
state	setting component state； (normal, active, disable)



nw	n	ne
w	center	e
sw	s	se

雨课堂 Rain Classroom

Basic Widgets

1. Label

```
import tkinter as tk
win = tk.Tk();
win.title("我的窗口")
lab1 = tk.Label(win, text = '你好', anchor= 'nw')     # Displaying text
lab1.pack()                                           # layout Label
lab2 = tk.Label(win, bitmap = 'question')             # Displaying build-in bitmap
lab2.pack()                                           # layout Label
bm = tk.PhotoImage(file = r'myimage.png')             # Displaying a image on disk
lab3 = tk.Label(win, image = bm)
lab3.pack()                                           # layout Label
win.mainloop()
```

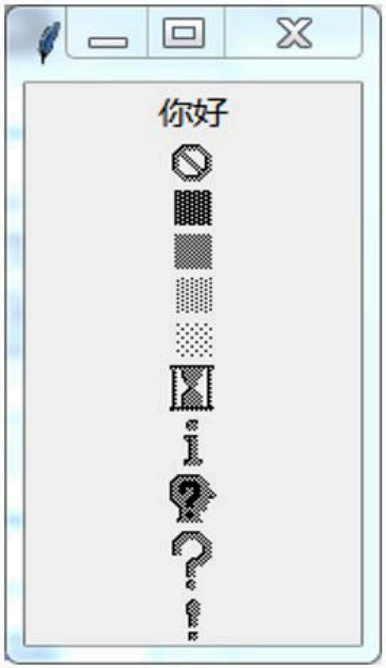


Basic Widgets

1. Label

```
import tkinter as tk
win = tk.Tk();
win.title("显示所有内置位图")
lab1 = tk.Label(win, text = '你好', anchor= 'nw')
lab1.pack()

# Displaying build-in bitmaps
bitmaps = ['error', 'gray75', 'gray50', 'gray25', 'gray12', 'hourglass', 'info',
'questhead', 'question', 'warning']
for bit_map in bitmaps:
    lab2 = tk.Label(win, bitmap = bit_map).pack()
win.mainloop()
```



Basic Widgets

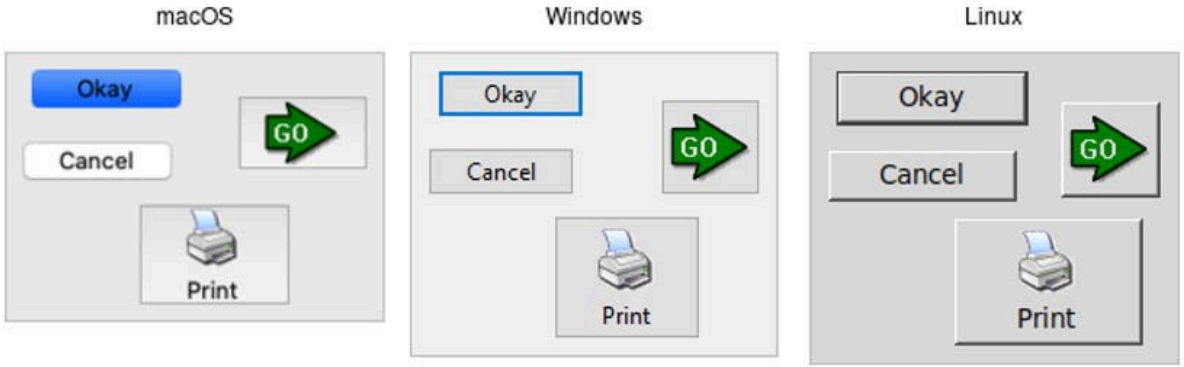
1. Label

options	Description
width, height	Specifies a desired width/height for the label.
compound	Specifies if the widget should display text and bitmaps/images at the same time， Must be one of the values none(default), bottom, top, left, right, or center. (图像，文本是否同时显示，如同时图像在文本的相对位置)
wraplength	Specifies the maximum line length (如果文本太长，多长换行)
justif	When there are multiple lines of text displayed in a widget, this option determines how the lines line up with each other. Must be one of left, center, or right. (指定多行的对齐方式)
anchor	Specifies how the information in a widget (e.g. text or a bitmap) is to be displayed in the widget.
image, bm	Show custom image， e.g. png, gif
bitmap	Show build-in bitmap

Basic Widgets

2. Button

- A button, unlike a label, is very much there to interact with.
- Users press a button to perform an action.
- Like labels, they can display text or images but accept additional options to change their behavior.



Basic Widgets

2. Button

```
import tkinter as tk
root = tk.Tk()
root.title("Button Test")
def callback():                    # define a Callback function
    tk.messagebox.showinfo("Python command", "人生苦短、我用Python")
tk.Button(root, text="外观装饰边界附近的标签", width=19, relief=GROOVE, bg="red").pack()
tk.Button(root, text="设置按钮状态", width=21, state=DISABLED).pack()
tk.Button(root, text="设置bitmap放到按钮左边位置", compound="left", bitmap="error").pack()
tk.Button(root, text="设置command事件调用命令", fg="blue", bd=10, width=28,
          command=callback).pack()
tk.Button(root, text ="设置高度宽度以及文字显示位置", anchor = 'sw', width = 30, height = 2).pack()
root.mainloop()
```



Basic Widgets

2. Button

options	Description
text	Specifies a string to be displayed inside the widget.
command	Specifies a Tcl command to associate with the button
compound	Specifies if the widget should display text and bitmaps/images at the same time (图像/文本是否同时显示，如同时图像在文本的相对位置)
bitmap	Specifies a bitmap to display in the widget
TakeFocus	Determines whether the window accepts the focus during keyboard traversal
bg, fg	Specifies the normal background/foreground color to use when displaying the widget.
activebackground, activeforeground	Specifies background/foreground color to use when drawing active elements. An element is active if the mouse cursor is positioned over the element and pressing a mouse button will cause some action to occur.
underline	Specifies the integer index of a character to underline in the widget. (快捷键的索引)

雨课堂 Rain Classroom

Basic Widgets

2. Button

- The button command creates a new Tcl command, This command may be used to invoke(调用) various operations on the widget.
  - Note: The following commands are ignored if the button's state is disabled.

COMMAND	Description
flash()	Flash the button. This is accomplished by redisplaying the button several times, alternating between the configured activebackground and background colors. At the end of the flash the button is left in the same normal/active state as when the command was invoked.
invoke()	Invoke the Tcl command associated with the button, if there is one. The return value is the return value from the Tcl command, or an empty string if there is no command associated with the button.

Basic Widgets

3. Entry (单行文本框)

```
import tkinter as tk
def  btnHelloClicked():                # callback function
    cd = float(entryCd.get())          # Get the contents in the entry and convert it to a fp number
    labelHello.config(text = "%.2f °C = %.2f °F" % (cd, cd*1.8+32))

root = tk.Tk()
root.title("Entry Test")
labelHello = tk.Label(root, text="转换°C to °F", height=5, width=20, fg="blue")
labelHello.pack()
entryCd = tk.Entry(root).pack()  # Create a Entry component and layout
btnCal = tk.Button(root, text="转换温度", command=btnHelloClicked)  #按钮
btnCal.pack()
root.mainloop()
```


雨课堂 Rain Classroom

Basic Widgets

3. Entry (单行文本框)

```
import tkinter as tk
def  btnHelloClicked():                    # callback function
    cd = float(entryCd.get())              # Get the contents in the entry and convert it to a fp number
    labelHello.config
                                                    f °F" %
root = tk.Tk()
root.title("Entry T
labelHello = tk.Lab                                 o °F", h                    'blue")
labelHello.pack()
entryCd = tk.Entry                                  a Entry
btnCal = tk.Button(root, text="转换温度", command=btnHelloClicked)  #按钮
btnCal.pack()
root.mainloop()
```





雨课堂 Rain Classroom

Basic Widgets

3. Entry (单行文本框)

- Configuration Options
  - All widgets have several configuration options that control how the widget is displayed or how it behaves.

```
import tkinter as tk
def  btnHelloClicked():                # callback function
    cd = float(entryCd.get())          # Get the contents in the entry and convert it to a fp number
    labelHello.config(text = "%.2f °C = %.2f °F" %(cd, cd*1.8+32))

# 组件属性除了可以用.config修改，还可以用以下方法修改：
    labelHello['text'] = "%.2f °C = %.2f °F" %(cd, cd*1.8+32)
```


雨课堂 Rain Classroom

Basic Widgets

3. Entry (单行文本框)

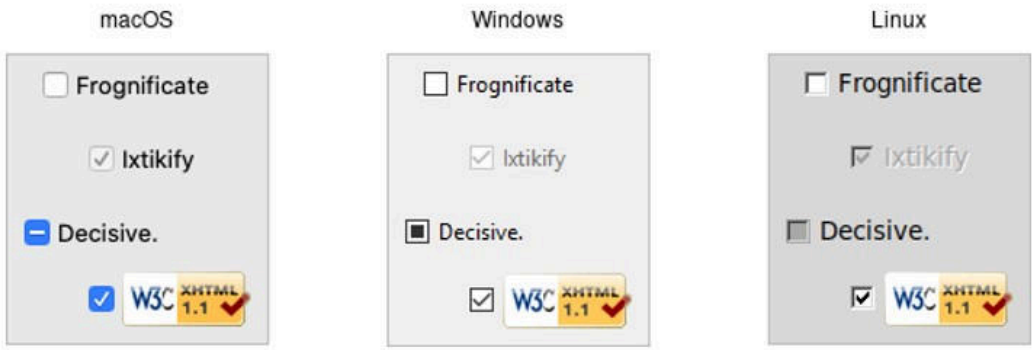
```
import tkinter as tk
def  btnHelloClicked():                  # callback function
   cd = float(textE.get())               # Get the contents in the entry and convert it to a fp number
   textL.set("%.2f °C = %.2f °F" % (cd, cd*1.8+32))

root = tk.Tk()
root.title("Entry Test")
textL = StringVar(value = "转换℃ to ℉" )
textE = StringVar(value = "请输入温度值")
tk.Label(root, textvariable = textL, height=5, width=20, fg="blue").pack()
tk.Entry(root, textvariable = textE).pack()          # textvariable选项可以将该组件的文本链接到一个变量，
                                                     # 当变量发生变化的时候，该组件自动更新该文本
tk.Button(root, text="转换温度", command = btnHelloClicked).pack()
root.mainloop()
```

Basic Widgets

4. Checkbutton (复选按钮)

- A checkbutton widget is like a regular button that also holds a binary value of some kind (i.e., a toggle).
- When pressed, a checkbutton flips the toggle and then **invokes its callback**.
- Checkbutton widgets are frequently used to allow users to turn an option on or off.



Basic Widgets

4. Checkbutton (复选按钮)

```
import tkinter as tk
def  checkButtonClicked():                    # callback function
    if checkV.get() == 0:     textL.set("check button is select" )
    elif checkV.get() == 1:  textL.set("check button is not select" )
    else:                          textL.set("check button is uncertain" )
root = tk.Tk()
root.title("Entry Test")
textL = StringVar(value = "test" )
checkV = IntVar()
tk.Label(root, textvariable = textL, height=5, width=20, fg="blue").pack()
tk.Checkbutton(root, text = "cb Test", variable=checkV, onvalue=0, offvalue=1,
                    command = checkButtonClicked).pack()
root.mainloop()
```


雨课堂 Rain Classroom

Basic Widgets

4. Checkbutton (复选按钮)

options	Description
variable	Specifies the name of a global variable to set to indicate whether or not this button is selected. Defaults to the name of the button within its parent
onvalue	Specifies value to store in the button's associated variable whenever this button is selected. Defaults to "1".
offvalue	Specifies value to store in the button's associated variable whenever this button is deselected. Defaults to "0".
command	Specifies a Tcl command to associate with the button. This command is typically invoked when mouse button 1 is released over the button window. The button's global variable will be updated before the command is invoked.

Basic Widgets

5. Radiobutton (单选按钮)

```
import tkinter as tk
def  RBClicked():                    # callback function
   textL.set(f"RadioButton value = {r.get()}")
root = tk.Tk()
root.title("使用单选按钮的例子")
r = tk.IntVar(value = 1)
textL = tk.StringVar(value = "test" )
tk.Label(root, textvariable=textL, height=5, width=20, fg="blue").pack()
tk.Radiobutton(root, variable=r, value=1, text='中国', command=RBClicked).pack()
tk.Radiobutton(root, variable=r, value=2, text='美国', command=RBClicked).pack()
tk.Radiobutton(root, variable=r, value=3, text='日本', command=RBClicked).pack()
tk.Radiobutton(root, variable=r, value=4, text='加拿大', command=RBClicked).pack()
tk.Radiobutton(root, variable=r, value=5, text='韩国', command=RBClicked).pack()
root.mainloop()
```

Basic Widgets

Example

```
import tkinter as tk
def colorChecked():
    label_1.config(fg = color.get())

def typeChecked():
    textType = typeBold.get() + typeItalic.get()
    if textType == 1:
        label_1.config(font = ("Arial", 12, "bold"))
    elif textType == 2:
        label_1.config(font = ("Arial", 12, "italic"))
    elif textType == 3:
        label_1.config(font = ("Arial", 12, "bold italic"))
    else :
        label_1.config(font = ("Arial", 12))
```



Basic Widgets

Example

```
root = tk.Tk()
root.title("Radio & Check Test")
color = tk.StringVar(value = "blue")          # 三个颜色Radiobutton定义了同样的变量color
typeBold = tk.IntVar()                        # 定义了typeBlod变量表示文字是否为粗体
typeItalic = tk.IntVar()                      # 定义了typeItalic变量表示文字是否为斜体
tk.Label(root, text = "Check the format of text.", height = 3, font=("Arial", 12), fg = "blue").pack()
tk.Radiobutton(root, text="红色", variable=color, value="red", command=colorChecked).pack()
tk.Radiobutton(root, text="蓝色", variable=color, value="blue", command=colorChecked).pack()
tk.Radiobutton(root, text="绿色", variable=color, value="green", command=colorChecked).pack()
tk.Checkbutton(root, text="粗体", variable=typeBold, onvalue=1, offvalue=0,
               command=typeChecked).pack()
tk.Checkbutton(root, text ="斜体", variable=typeItalic, onvalue=2, offvalue=0,
               command = typeChecked).pack()
root.mainloop()
```


雨课堂 Rain Classroom

More Widgets

- This section introduces several more widgets: listbox, scrollbar, text, scale, spinbox, and progressbar.

Name	Description
Listbox	displays a list of single-line text items, usually lengthy, and allows users to browse through the list, selecting one or more. (列表框)
Scrollbar	helps users see all parts of another widget, whose content is typically much larger than what can be shown in the available screen space.
Text	provides users with an area so that they can enter multiple lines of text.
Scale	allows users to choose a numeric value through direct manipulation.(拖拽式取值)
Spinbox	allows users to choose numbers (or, in fact, items from an arbitrary list). (滚动式取值)
Progressbar	provides feedback to users about the progress of a lengthy operation.(进度条)

雨课堂 Rain Classroom

More Widgets

1. Listbox

- A listbox widget displays a list of single-line text items, usually lengthy, and allows users to browse through the list, selecting one or more.
- A *height* configuration option can specify the number of lines the listbox will display at a time without scrolling:

```
l = Listbox(parent, height=10)
```



More Widgets

1. Listbox

- Populating the Listbox Items:

```
import tkinter as tk
root = tk.Tk()
root.title("填充列表框项的例子")
choices = ["apple", "orange", "banana"]
choicesVar = tk.StringVar(value=choices)
l = tk.Listbox(root, listvariable=choicesVar).pack()# Create a listbox and assign initial value
choices.append("peach")
choicesVar.set(choices)    # Modify the variables of the listbox, the listbox is automatically updated
root.mainloop()
```


雨课堂 Rain Classroom

More Widgets

1. Listbox

```
import tkinter as tk
root = tk.Tk()
root.title("使用Listbox组件的例子")

m = tk.StringVar(value = ['北京','天津','上海'])        # 给listbox创建一个字符串变量对象
l = tk.StringVar(value = "显示Button状态")              # 给label创建一个字符串变量对象

def callbutton1():
    l.set(m.get())                                      # 获取列表内容，并在label上显示

def callbutton2():
    t = ""
    for i in lb.curselection():                         # 选中item的索引使用curselection获取
        t += f"{lb.get(i)} "                            # 逐项获得选中item，并连接成一个字符串
    l.set(t)
```



More Widgets

1. Listbox

```
lb = tk.Listbox(root, listvariable=m, selectmode=tk.MULTIPLE).pack()

tk.Label(root, textvariable=l, bg = "yellow").pack()

tk.Button(root, text = '获取Listbox的所有内容', command=callbutton1).pack()
tk.Button(root, text = '获取Listbox的选中内容', command=callbutton2).pack()

root.mainloop()
```



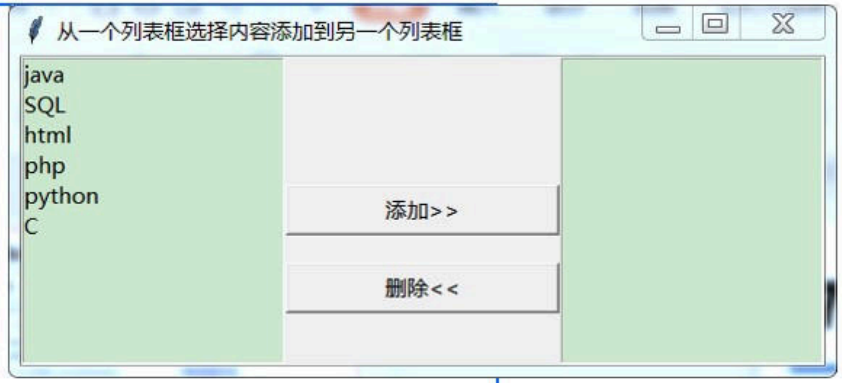
More Widgets

1. Listbox

```
import tkinter as tk
root = tk.Tk()
root.title("从一个列表框选择内容添加到另一个列表框")

def callbutton1():
    for i in listb1.curselection():      # 遍历选中项
        listb2.insert(0, listb1.get(i))  #添加到右侧列表框

def callbutton2():
    for i in listb2.curselection():      #遍历选中项
        listb2.delete(i)                 #从右侧列表框中删除
```



More Widgets

1. Listbox

```
li = ['C', 'python', 'php', 'html', 'SQL', 'java']
listb1 = tk.Listbox(root)                    #创建两个列表框组件
for item in li:                              #左侧列表框组件插入数据
    listb1.insert(0, item)
listb1.grid(row=0, column=0, rowspan=2) # 将列表框组件放置到窗口对象中

listb2 = tk.Listbox(root)
listb2.grid(row=0, column=2, rowspan=2)

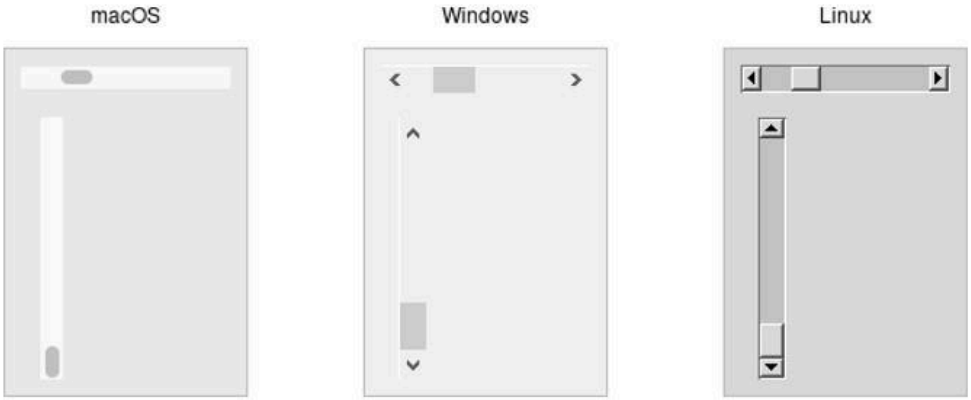
tk.Button (root, text = '添加>>', command=callbutton1, width=20).grid(row=0, column=1)
tk.Button (root, text = '删除<<', command=callbutton2, width=20).grid(row=1, column=1)
root.mainloop()
```

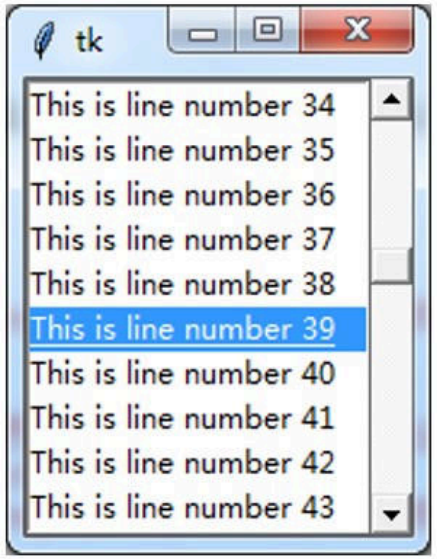

雨课堂 Rain Classroom

More Widgets

2. Scrollbar

- A scrollbar widget helps users see all parts of another widget, whose content is typically much larger than what can be shown in the available screen space.





More Widgets

2. Scrollbar

- Unlike in some user interface toolkits, Tk scrollbars are not a part of another widget (e.g., a listbox) but are a separate widget altogether.
- Instead, scrollbars communicate with the scrolled widget by calling methods on the scrolled widget(.yview); as it turns out, the scrolled widget also needs to call methods on the scrollbar(.set).

```
s = tk.Scrollbar( parent, orient=VERTICAL, command=listbox.yview)
listbox.configure(yscrollcommand=s.set)
```


雨课堂 Rain Classroom

More Widgets

2. Scrollbar

```
import tkinter as tk
root = tk.Tk()
l = []
for line in range(100):                    #列表框内追加100项内容
  l.append(f"This is line number {line}")
listVar = tk.StringVar(value = l)
mylist = tk.Listbox(root, listvariable=listVar, yscrollcommand = scrollbar.set)        # 创建列表框
mylist.pack( side = LEFT, fill = BOTH )
scrollbar = tk.Scrollbar(root, command = mylist.yview)
scrollbar.pack( side = RIGHT, fill=Y )
mylist.config(yscrollcommand = scrollbar.set)
root.mainloop()
```

■ 不能在此设置列表框与滚动条的连接，因为滚动条还没有定义。

雨课堂 Rain Classroom

More Widgets

2. Scrollbar

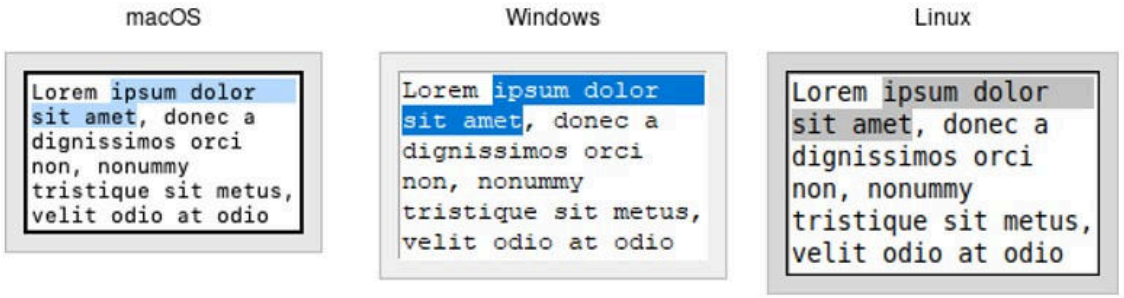
```
import tkinter as tk
def print_item(event):                          # 鼠标松开事件打印出当前选中项内容
    print (mylist.get(mylist.curselection()))
root = tk.Tk()
mylist = tk.Listbox(root)                       # 创建列表框
mylist.bind('<ButtonRelease-1>', print_item)    # 绑定鼠标松开事件
for line in range(100):                         # 列表框内追加100项内容
   mylist.insert(tk.END, f"This is line number {line}")
mylist.pack( side = tk.LEFT, fill = BOTH )
scrollbar = tk.Scrollbar(root, command = mylist.yview)
scrollbar.pack( side = tk.RIGHT, fill=tk.Y )
mylist.config(yscrollcommand = scrollbar.set)
root.mainloop()
```


雨课堂 Rain Classroom

More Widgets

3. Text

- A text widget provides users with an area so that they can enter multiple lines of text. Text widgets are part of the classic Tk widgets, not the themed Tk widgets.



- Tk's text widget is, along with the canvas widget, one of two uber-powerful widgets that provide amazingly deep but easily programmed features. Text widgets have formed the basis for full word processors, outliners, web browsers, and more.

More Widgets

3. Text

- Text widgets are created using the Text class:

```
txt = Text(parent, width=40, height=10)
```

  - The *width* and *height* options specify the requested screen size of the text widget, in characters and rows, respectively.

- A text widget can be disabled so that no editing can occur.

```
txt['state'] = 'disabled'
```


雨课堂 Rain Classroom

More Widgets

3. Text

- Text widgets are created using the Text class:

```
txt = Text(parent, width=40, height=10)
```

  - The *width* and *height* options specify the requested screen size of the text widget, in characters and rows, respectively.

- A text widget can be disabled so that no editing can occur.

```
txt['state'] = 'disabled'
```


雨课堂 Rain Classroom

More Widgets

3. Text

```
import tkinter as tk
root = tk.Tk()
photo = PhotoImage(file=r'C:\Users\Administrator\wechat.png')
def show_text():
    mytext.insert(END, "show a new image")
    mytext.image_create(END,image = photo)          # 可以在Text组件内放置嵌入式图像

mytext = tk.Text(root,width=50, height=40)          # 创建Text组件，宽50字符，高40行
mytext.pack()
mytext.insert(INSERT,"starting writing\n")          # 插入一段文字，INSERT是索引位置
button = tk.Button(mytext, text='Click me!', command=show_text)

mytext.window_create(INSERT, window=button)  # 可以在Text组件内放置任意的嵌入式窗口(例如：按钮)
root.mainloop()
```


雨课堂 Rain Classroom

More Widgets

3. Text

- Text of Tkinter provides a range of different *index* types:

Name	Description
"line.char"	Indicates *char*'th character on line *line*.
"line.end"	Indicates the last character on line *line*.
INSERT	Indicates the position of the cursor
CURRENT	Indicates the closest position to the mouse coordinates.(鼠标坐标最接近的位置)
END	Indicates the end of the text (the character just after the last newline).
marks	Marks are used for remembering particular places in a text. Only a single position may be associated with a mark at any given time. the mark *insert* is associated with the insertion cursor, the mark *current* is associated with the character closest to the mouse (缺省有两个标记，insert和current，其他可以自定义)
tag	A tag is a textual string that is associated with some of the characters in a text. Tags may contain arbitrary characters.Tags are used to specify attributes of strings associated with them.

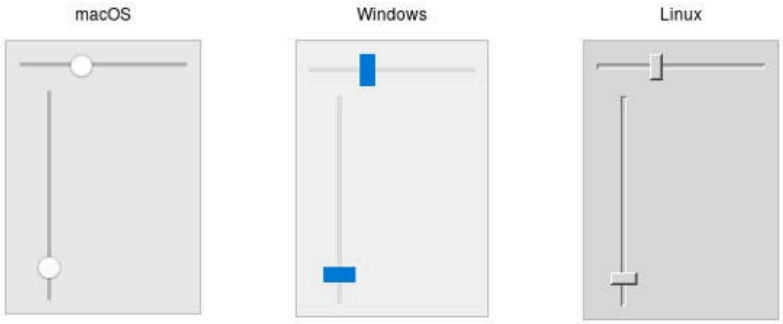
雨课堂 Rain Classroom

More Widgets

4. Scale

- A scale widget allows users to choose a numeric value through direct manipulation.

```
s = tk.Scale(parent, orient=HORIZONTAL, length=200, from_=1.0, to=100.0)
```



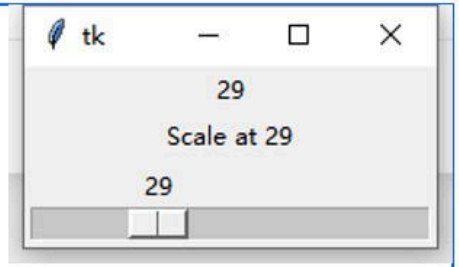
■ 因为from在Python中是保留字，所以需要加个下划线。

- The *orient* option may be either *horizontal* or *vertical*.（方向）
- The *length* option is specified in screen units (e.g., pixels).（显示长度）
- The *from/to* option define the range of the number that the scale allows users to choose.(数值范围)

More Widgets

4. Scale

```
import tkinter as tk
root = tk.Tk()
root.title("Example for Scale")
num = StringVar()
# 此label与scale共用一个变量，自动更新数据
tk.Label(root, textvariable=num).grid(column=0, row=0, sticky='we')
manual = tk.Label(root)                    # 这个label通过scale的回调函数手动更新数据
manual.grid(column=0, row=1, sticky='we')
def update_lbl(val):
    manual['text'] = "Scale at " + val
scale = tk.Scale(root, orient='horizontal', length=200, from_=1.0, to=100.0, variable=num,
                 command=update_lbl)
scale.grid(column=0, row=2, sticky='we')
scale.set(20)
root.mainloop()
```

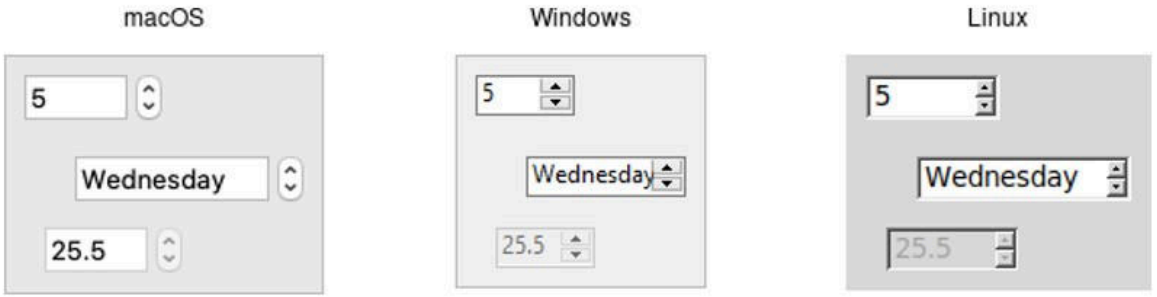


More Widgets

5. Spinbox

- A spinbox widget allows users to choose numbers (or, in fact, items from an arbitrary list任意列表).
- It does this by combining an entry-like widget showing the current value with a pair of small up/down arrows, which can be used to step through the range of possible choices.

```
spinval = StringVar()
s = ttk.Spinbox(parent, from_=1.0, to=100.0, textvariable=spinval)
```

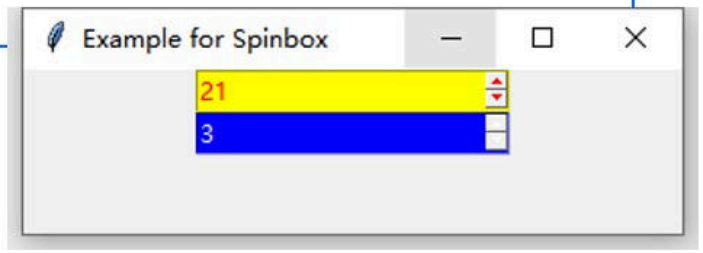


More Widgets

5. Spinbox

```
import tkinter as tk
root = tk.Tk()
root.title("Example for Spinbox")

s1 = tk.Spinbox(root, bg='yellow', fg='red', from_=1.0, to=100.0, increment=5)
s1.pack()
sV = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
s2 = tk.Spinbox(root, bg='yellow', fg='red', from_=1.0, to=100.0, values=sV)
s2.pack()
root.mainloop()
```

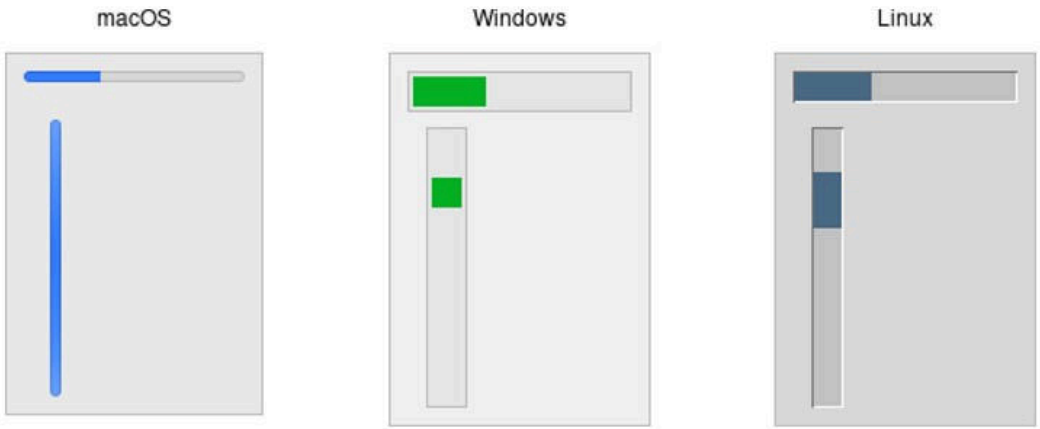


More Widgets

6. Progressbar

- A progressbar widget provides feedback to users about the progress of a lengthy(较长) operation.

```
p = tk.Progressbar(parent, orient=HORIZONTAL, length=200, mode='determinate')
```



More Widgets

6. Progressbar

- In situations where you can estimate how long the operation will take to complete
  - you can display what fraction has already been completed. (确定提示完成度：0%-100%)
  - Otherwise, you can indicate the operation is continuing, but without suggesting how much longer it will take.(不确定提示完成度，提示正在处理，进度条来回往复)

```
p = tk.Progressbar(parent, orient=HORIZONTAL, length=200, mode='determinate'/'indeterminate')
```


雨课堂 Rain Classroom

More Widgets

6. Progressbar

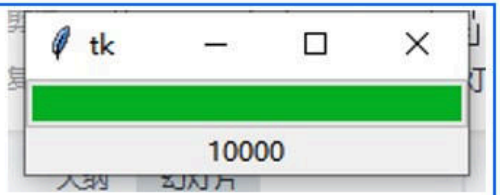
```
import tkinter as tk
from tkinter import ttk                    # Progressbar在ttk模块中
def increment():
    p1.start()
def stop():
    p1.stop()
root = tk.Tk()
root.geometry='320x240'
p1 = ttk.Progressbar(root, length=200, mode="indeterminate", maximum=200,
                     orient=tk.HORIZONTAL)
p1.grid(row=1, column=1)
ttk.Button(root, text="Start", command=increment).grid(row=1, column=0)
ttk.Button(root, text="Stop", command=stop).grid(row=1, column=2)
root.mainloop()
```



More Widgets

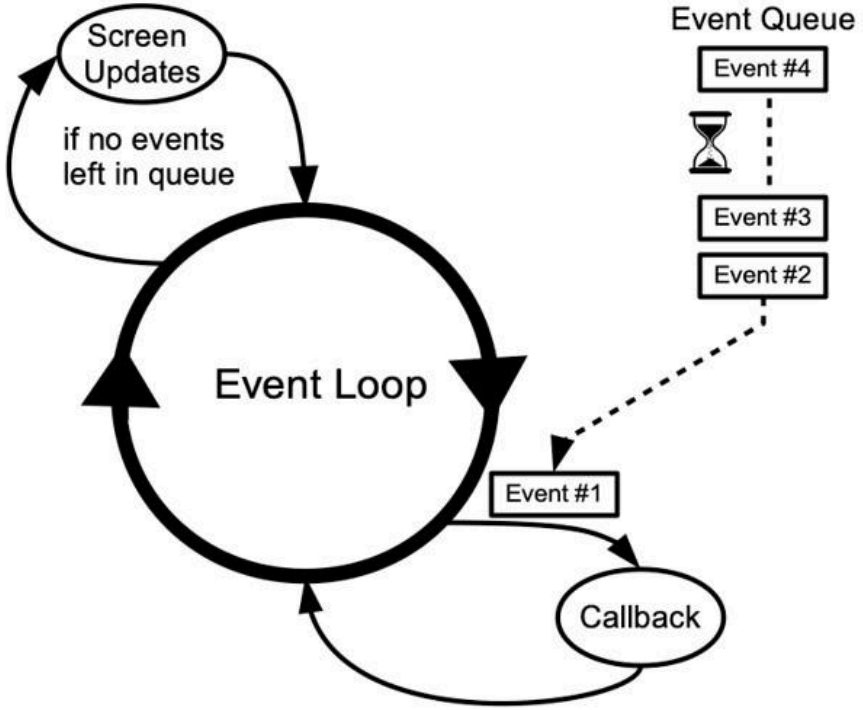
6. Progressbar

```
import tkinter as tk
from tkinter import ttk                    # Progressbar在ttk模块中
root = tk.Tk()
root.geometry='320x240'
proVar = IntVar()
p1 = ttk.Progressbar(root, length=200, mode="determinate", maximum=200, variable=proVar,
orient=tk.HORIZONTAL)
p1.grid(row=0, column=0)
ttk.Label(root, textvariable=str(proVar)).grid(row=3, column=0)
for i in range(10000000):
    if i%1000==0:
        proVar.set(proVar.get()+1)
root.mainloop()
```



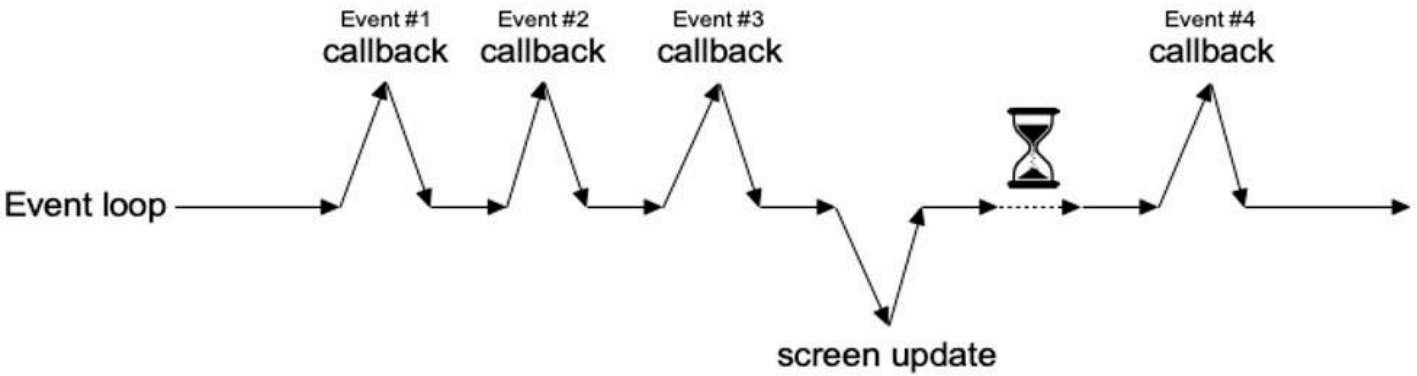
Event Loop

- After we construct(构建) an application's initial user interface, it enters the **Tk event loop.** The event loop continually processes events, pulled from the system event queue, usually dozens of times a second.
- It watches for mouse or keyboard events, invoking command callbacks and event bindings as needed.
- All **screen updates** are processed only in the event loop



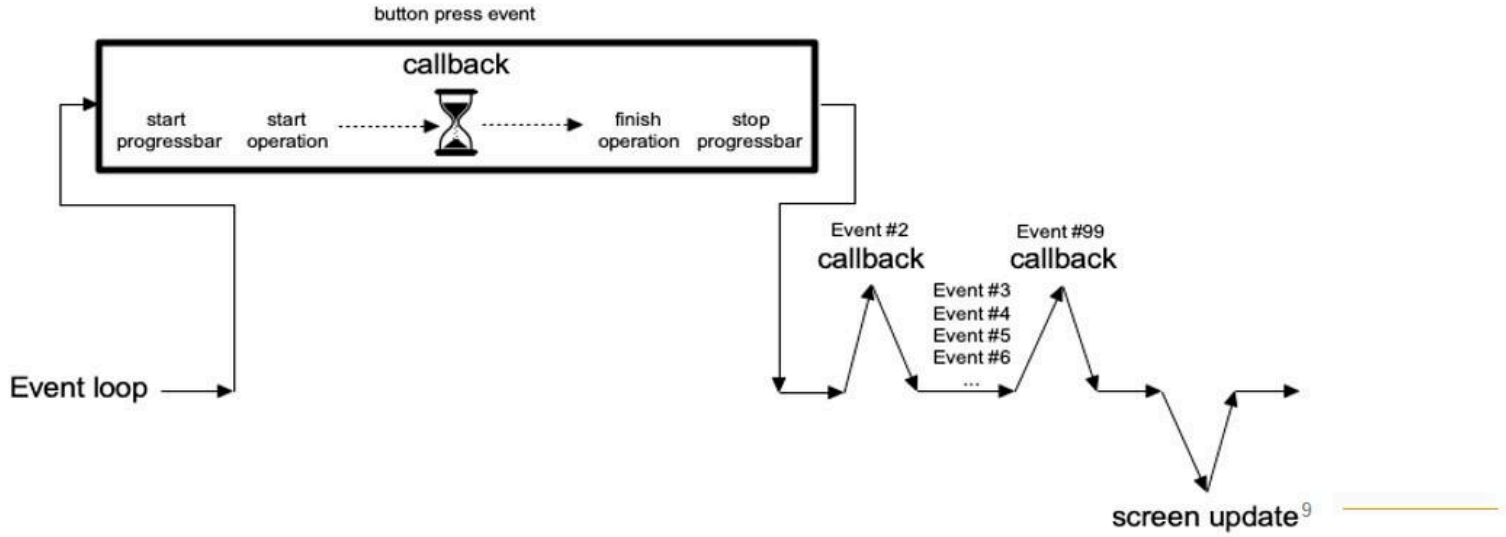
Event Loop

- In a normal situation, each deviation(时间差) from the event loop (callback, screen update) takes only a fraction of a second before returning control to the event loop.



Event Loop

- In **progressbar** scenario(场景), the whole thing probably started from an event (for loop). So the event loop calls our application code to handle the event. Only code return control back to the event loop. No events have been processed in the meantime, and no screen redrawing has occurred. Events have been piling up in the event queue.



Event Loop

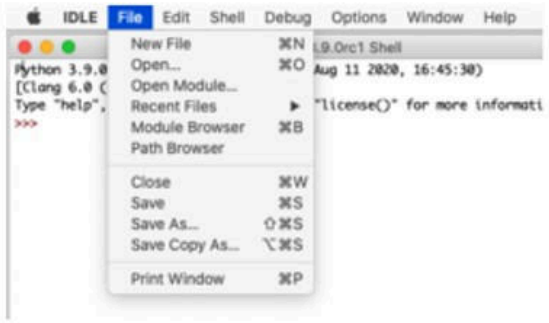
```
import tkinter as tk
from tkinter import ttk                    # Progressbar在ttk模块中
root = tk.Tk()
root.geometry='320x240'
proVar = IntVar()
def showpro():
    proVar.set(proVar.get()+1)             # 进度条变量proVar更新
    if proVar.get()<200:
        root.after(100,showpro)            # 为了能够持续调用showpro，需要嵌套调用root.after
p1 = ttk.Progressbar(root, length=200, mode="determinate", maximum=200, variable=proVar,
                     orient=tk.HORIZONTAL)
p1.grid(row=0, column=0)
ttk.Label(root, textvariable=str(proVar)).grid(row=3, column=0)
root.after(100, showpro)                   # 使用非阻塞方式调用函数showpro，100ms之后调用
root.mainloop()
```


雨课堂 Rain Classroom

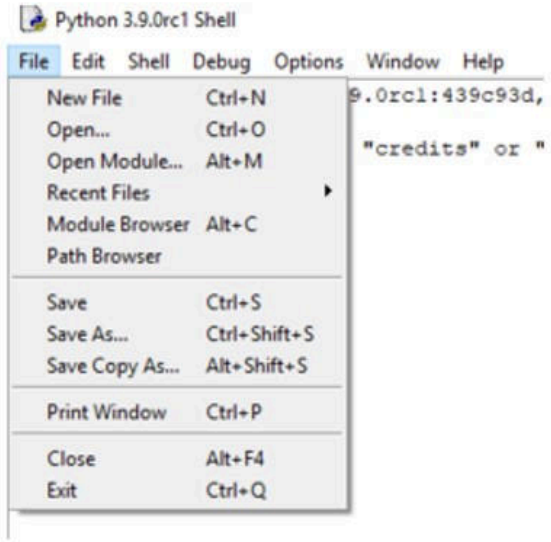
Menus

- This section describes how to handle menubars(菜单栏) and popup menus(上下文菜单) in Tk.

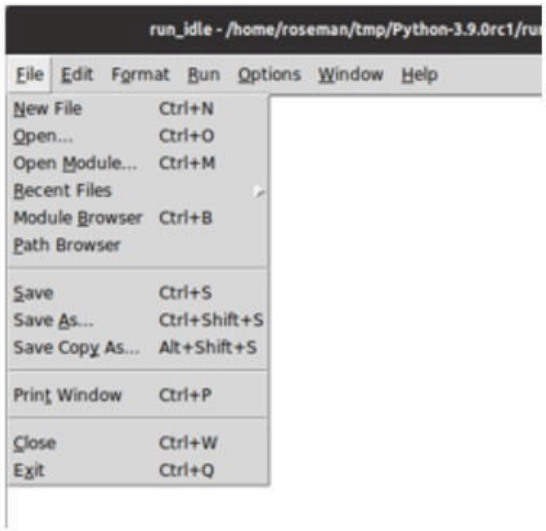
macOS



Windows



Linux



雨课堂 Rain Classroom

Menus

1. Create and show a Menu

```
import tkinter as tk
root = tk.Tk()
def FileCB():                    # menu callback function
    print("You clicked the menu_File!")
def EditCB():                    # menu callback function
    print("You clicked the menu_Edit!")
def ViewCB():                    # menu callback function
    print("You clicked the menu_View!")
topmenu = tk.Menu(root)
topmenu.add_command(label ='文件', command = FileCB)
topmenu.add_command(label ='编辑', command = EditCB)
topmenu.add_command(label ='视图', command = ViewCB)
root['menu'] = topmenu           # 附加主菜单到窗口
root.mainloop()
```



Menus

2. Submenus

```
parent_Menu = tk.Menu(root)
subMenu = tk.Menu(parent_Menu)
parent_Menu.add_cascade(label = parentMenuText, menu = subMenu)
```





Menus

2. Submenus

```
import tkinter as tk
def hello():
    print("I'm a child menu")
root = tk.Tk()
topmenu = tk.Menu(root)                  # 创建主菜单
filemenu = tk.Menu(topmenu)              # 创建下拉菜单，指明是topmenu的下拉菜单
editmenu = tk.Menu(topmenu)              # 创建下拉菜单，指明是topmenu的下拉菜单
for item in ['打开','关闭','退出']:       # 添加下拉菜单项
    filemenu.add_command(label =item, command = hello)
for item in ['复制','剪切','粘贴']:       # 添加下拉菜单项
    editmenu.add_command(label =item, command = hello)
topmenu.add_cascade(label ='文件', menu = filemenu)    # 把 filemenu 作为文件下拉菜单
topmenu.add_cascade(label ='编辑', menu = editmenu)    # 把 editmenu 作为编辑下拉菜单
root['menu'] = topmenu                   # 附加主菜单到窗口
root.mainloop()
```

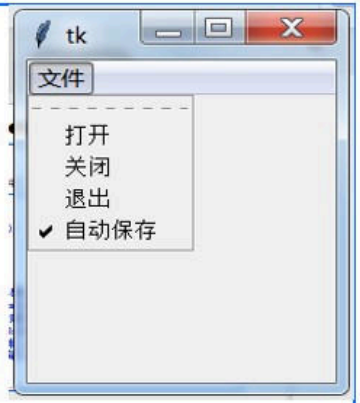


雨课堂 Rain Classroom

Menus

3. Checkbutton and Radiobutton Items

```
import tkinter as tk
def hello():
    print(save.get())
root = tk.Tk()
save = tk.StringVar()
radio = tk.StringVar()
topmenu = tk.Menu(root)
filemenu = tk.Menu(topmenu)
topmenu.add_cascade(label='文件', menu = filemenu)
for item in ['打开','关闭','退出']:
    filemenu.add_command(label =item, command = hello)
filemenu.add_checkbutton(label='自动保存', command=hello, variable=save, onvalue=1, offvalue=0)
filemenu.add_radiobutton(label='One', variable=radio, value=1)
filemenu.add_radiobutton(label='Two', variable=radio, value=2)
root['menu'] = topmenu
root.mainloop()
```



Menus

4. Separators

```
import tkinter as tk
def hello():
    print("I'm a child menu")
root = tk.Tk()

topmenu = tk.Menu(root)
filemenu = tk.Menu(topmenu)
topmenu.add_cascade(label ='文件', menu = filemenu)
filemenu.add_command(label ='打开', command = hello)
filemenu.add_command(label ='关闭', command = hello)
filemenu.add_separator()            # '关闭'和'退出'之间添加分隔符
filemenu.add_command(label ='退出', command = hello)
root['menu'] = topmenu
root.mainloop()
```



Menus

5. Contextual Menus

- Contextual menus ("popup" menus) are typically invoked by a right mouse button click on an object in the application.
- To create a contextual menu, use exactly the same commands used to create a menubar.
- To activate the menu, we'll have to create an event binding to capture the click.

```
import tkinter as tk
root  = tk.Tk()
menu = tk.Menu(root)
menu.add_command(label = '剪切', command = hello1)
menu.add_command(label = '复制', command = hello2)
menu.add_command(label = '粘贴', command = hello3)
def popup(event)                                    # 事件处理函数
   menu.post(event.x_root, event.y_root)            # 在鼠标右键位置显示菜单
root.bind('<Button -3>', popup)                     # 绑定事件
```


雨课堂 Rain Classroom

Menus

5. Contextual Menus

```
import tkinter as tk
def  popup(event):                                  #右键事件处理函数
    menu. post( event.x_root, event.y_root)         #在鼠标右键位置显示菜单
def hello1():       print("我是剪切命令")            #菜单事件处理函数
def hello2():       print("我是复制命令")
def hello3():       print("我是粘贴命令")
root = tk.Tk()
root.geometry("300x150")
menu = tk.Menu(root)
menu.add_command(label ='剪切', command = hello1)
menu.add_command(label ='复制', command = hello2)
menu.add_command(label ='粘贴', command = hello3)
s=tk.StringVar(value="大家好，这是测试上下文菜单")
tk.Entry(root, textvariable=s).pack()               #创建Entry组件界面
root.bind('<Button-3>', popup)                      #绑定右键事件
root.mainloop()
```


雨课堂 Rain Classroom

Menus

5. Contextual Menus

```
import tkinter as tk
def hello1():       print("我是剪切命令")                    #菜单事件处理函数
def hello2():       print("我是复制命令")
def hello3():       print("我是粘贴命令")
root = tk.Tk()
root.geometry("300x150")
menu = tk.Menu(root)
menu.add_command(label ='剪切', command = hello1)
menu.add_command(label ='复制', command = hello2)
menu.add_command(label ='粘贴', command = hello3)
s=tk.StringVar(value="大家好，这是测试上下文菜单")
tk.Entry(root, textvariable=s).pack()                    #创建Entry组件界面
root.bind('<Button-3>', lamda event: menu.post(event.x_root, event.y_root))          #绑定右键事件
root.mainloop()
```


雨课堂 Rain Classroom

7.2.9 对话框

- 对话框用于与用户交互和检索信息。
- Tkinter模块中有一些子模块，如：messagebox（消息盒）、filedialog（文件对话框）、colorchooser（颜色对话框）、simpleDialog（简单对话框），这些子模块包含了一些通用的预定义对话框；

- 用户也可以**通过继承TopLevel创建自定义对话框**。

雨课堂 Rain Classroom

7.2.9 对话框

1. 文件对话框

- Tkinter的子模块filedialog包含用于打开文件对话框的函数**askopenfilename()**
- 文件对话框供用户选择某文件夹下的文件。语法格式如下：

```
askopenfilename(title = '标题', filetypes = [('所有文件', '.*'), ('文本文件'), '.txt')])
```

  - 其中，filetypes：文件过滤器，可以筛选某种格式文件
  - title：设置打开文件对话框的标题
- 还有文件保存对话框函数**asksavesfilename()**，语法格式如下：

```
asksavesfilename(title = '标题', initialdir = 'd:\mywork', initialfile = 'hello.py')
```

  - 其中，initialdir： 默认保存路径即文件夹，如 'd:\mywork'
  - initialfile： 默认保存的文件名，如 'hello.py'

雨课堂 Rain Classroom

7.2.9 对话框

- 1. 文件对话

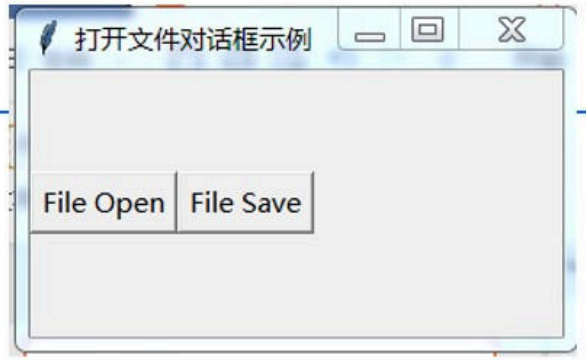
```
import tkinter as tk
from tkinter import filedialog
def openfile():                    #按钮事件处理函数
                                   #显示打开文件对话框，返回选中文件名以及路径
    r = tk.filedialog.askopenfilename(title='打开文件', filetypes=[('Python',
'*.py *.pyw'), ('All Files', '*.*')])
    print(r)
def savefile():                    #按钮事件处理函数
    #显示保存文件对话框
    r = tk.filedialog.asksaveasfilename(title='保存文件',
initialdir='d:\mywork', initialfile='hello.py')
    print(r)
```


雨课堂 Rain Classroom

7.2.9 对话框

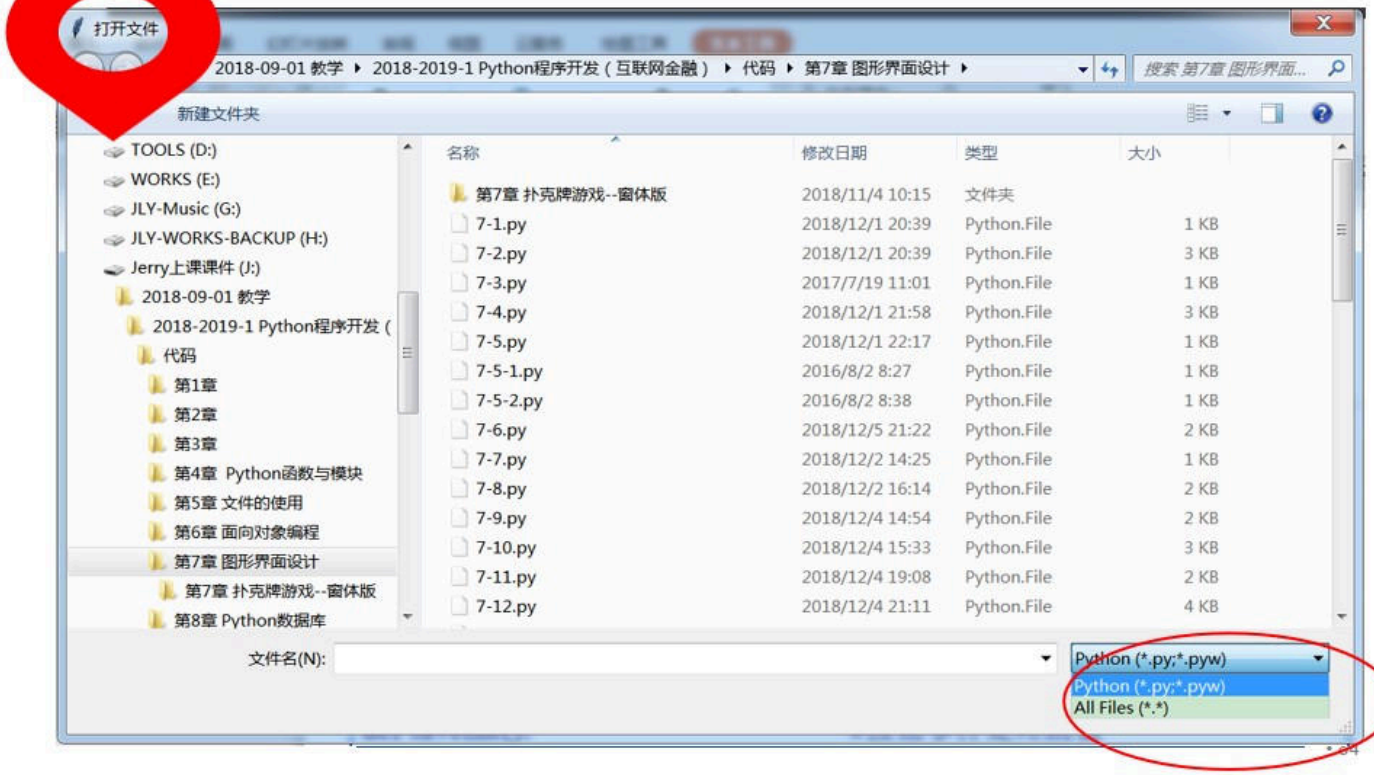
- 1. 文件对话

```
root = tk.Tk()
root.title('打开文件对话框示例')              # title属性用来指定标题
root.geometry("300x150")
btn1 = tk.Button(root, text='File Open', command=openfile) #创建Button组件
btn2 = tk.Button(root, text='File Save', command=savefile)  #创建Button组件
btn1.pack(side='left')
btn2.pack(side='left')
root.mainloop()
```



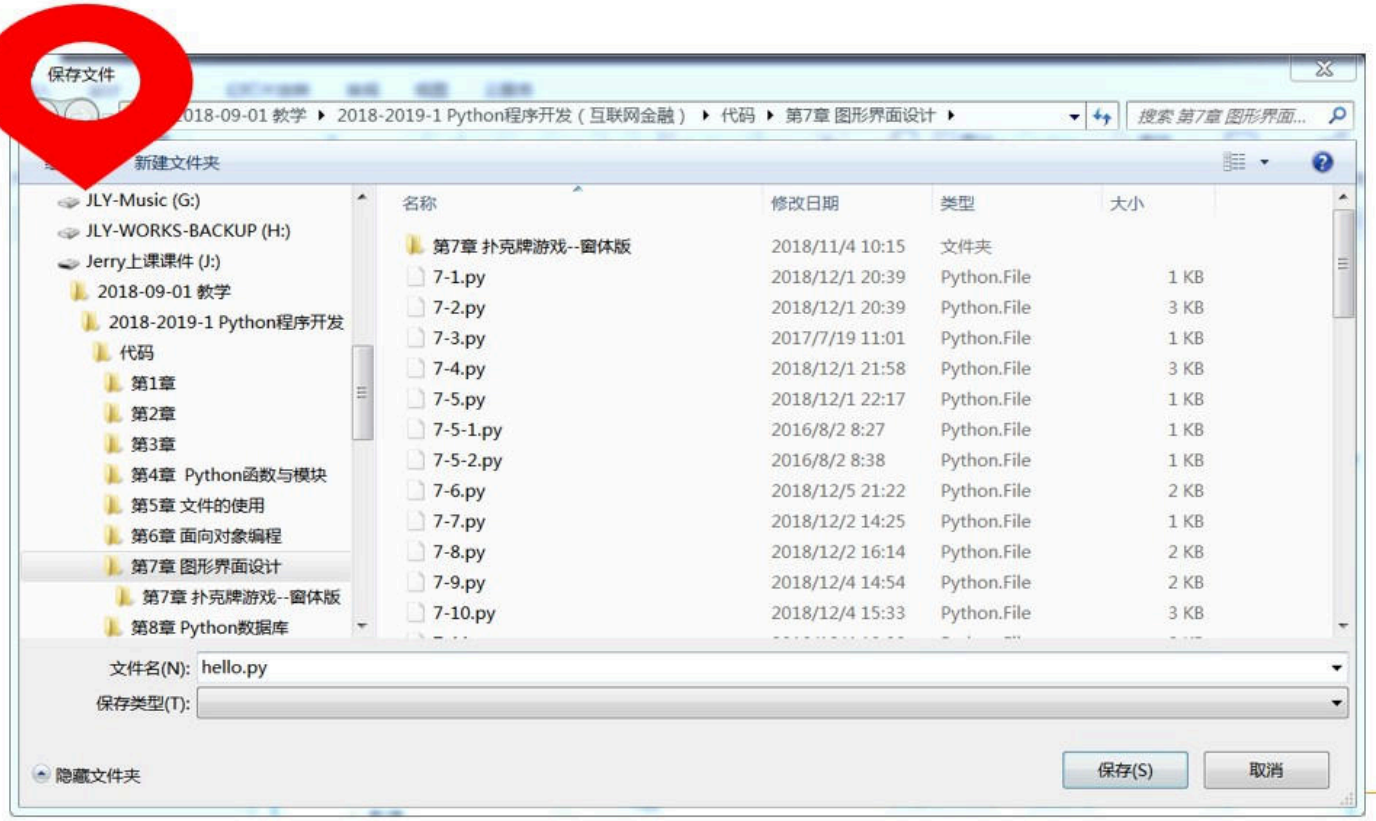
7.2.9 对话框

1. 文件对话框——实例



7.2.9 对话框

- 1. 文件


保存文件
2018-09-01 教学 ▸ 2018-2019-1 Python程序开发（互联网金融） ▸ 代码 ▸ 第7章 图形界面设计 ▸
搜索 第7章 图形界面...
新建文件夹
JLY-Music (G:)
JLY-WORKS-BACKUP (H:)
Jerry上课课件 (J:)
2018-09-01 教学
2018-2019-1 Python程序开发
代码
第1章
第2章
第3章
第4章 Python函数与模块
第5章 文件的使用
第6章 面向对象编程
第7章 图形界面设计
第7章 扑克牌游戏--窗体版
第8章 Python数据库
名称
修改日期
类型
大小
第7章 扑克牌游戏--窗体版 2018/11/4 10:15 文件夹
7-1.py 2018/12/1 20:39 Python.File 1 KB
7-2.py 2018/12/1 20:39 Python.File 3 KB
7-3.py 2017/7/19 11:01 Python.File 1 KB
7-4.py 2018/12/1 21:58 Python.File 3 KB
7-5.py 2018/12/1 22:17 Python.File 1 KB
7-5-1.py 2016/8/2 8:27 Python.File 1 KB
7-5-2.py 2016/8/2 8:38 Python.File 1 KB
7-6.py 2018/12/5 21:22 Python.File 2 KB
7-7.py 2018/12/2 14:25 Python.File 1 KB
7-8.py 2018/12/2 16:14 Python.File 2 KB
7-9.py 2018/12/4 14:54 Python.File 2 KB
7-10.py 2018/12/4 15:33 Python.File 3 KB
文件名(N): hello.py
保存类型(T):
隐藏文件夹
保存(S)
取消

7.2.9 对话框

2. 颜色对话框

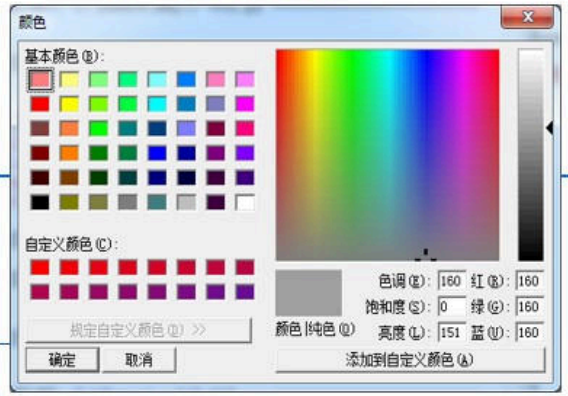
- Tkinter的子模块colorchooser包含用于打开颜色对话框的函数askcolor()
- 颜色对话框供用户选择某颜色。

```
import tkinter as tk
from tkinter import colorchooser
root = tk.Tk()
#调用askcolor返回选中颜色的(R,G,B)值和'#RRGGBB'的十六进制表示
color = colorchooser.askcolor()
print(color)
root.mainloop()
```

- 弹出颜色对话框，点击确定，关闭对话框，返回颜色值
- 如果点击取消，关闭对话框，返回（None，None）

输出结果：

((205.80078125, 5.01953125, 50.1953125), '#cd0532')



7.2.9 对话框

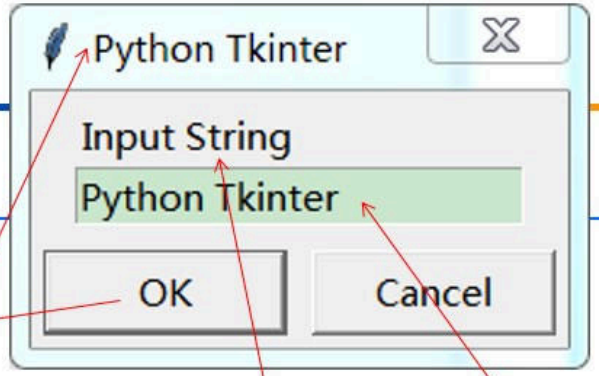
3. 简单对话框

- Tkinter的子模块simpledialog包含用于打开输入对话框的函数
  - **askfloat(title, prompt, 选项)：打开输入对话框，输入并返回浮点数**
  - **askinteger(title, prompt, 选项)：打开输入对话框，输入并返回整数**
  - **askstring(title, prompt, 选项)：打开输入对话框，输入并返回字符串**
- 其中，
  - title为窗口标题
  - prompt为提示文本信息
  - 选项包括：initialvalue（初始值）、minvalue（最小值）、maxvalue（最大值）

雨课堂 Rain Classroom

7.2.9 对话框

3. 简单对话框

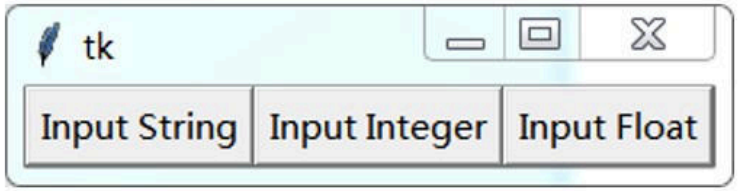


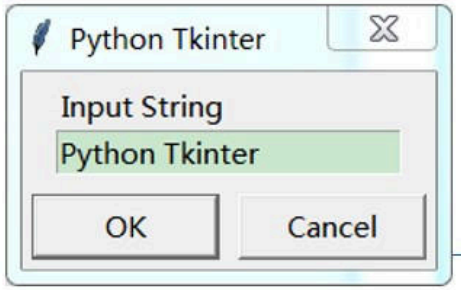

```
import tkinter as tk
from tkinter import simpledialog
def inputStr():
    r = simpledialog.askstring('Python Tkinter', 'Input String', initialvalue =
'Python Tkinter')
    print(r)
def inputInt():
    r = simpledialog.askinteger('Python Tkinter', 'Input Integer')
    print(r)
def inputFloat():
    r = simpledialog.askfloat('Python Tkinter', 'Input Float')
    print(r)
```

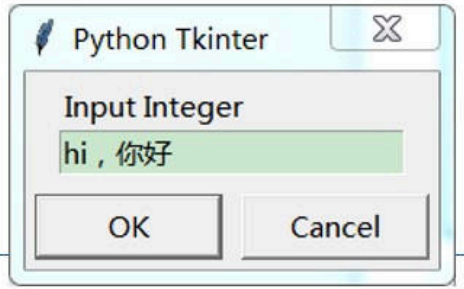
7.2.9 对话框

3. 简单对话框

```
root = tk.Tk()
btn1 = tk.Button(root, text='Input String', command=inputStr)
btn2 = tk.Button(root, text='Input Integer', command=inputInt)
btn3 = tk.Button(root, text='Input Float', command=inputFloat)
btn1.pack(side='left')
btn2.pack(side='left')
btn3.pack(side='left')
root.mainloop()
```





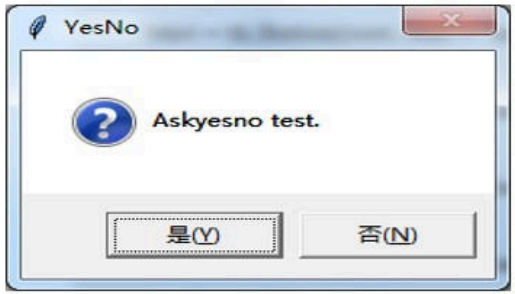




7.2.10 消息窗口（消息框）

- 消息窗口（messagebox）用于弹出提示框向用户进行告警，或让用户选择下一步如何操作。
- 消息框包括很多类型，常用的有info、warning、error、yesno、okcancel等，包含不同的图标、按钮以及弹出提示音。

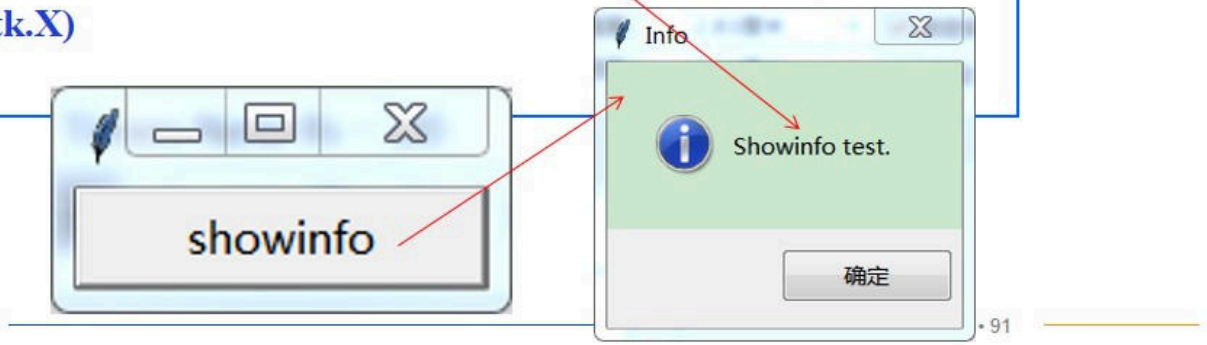




7.2.10 消息窗口（消息框）

Info消息窗口

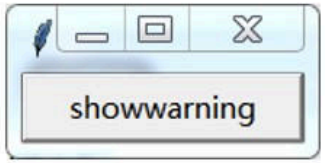
```
import tkinter as tk
from tkinter import messagebox as msgbox
def btn1_clicked():
    msgbox.showinfo("Info", "Showinfo test.")
root = tk.Tk()
root.title("MsgBox Test")
btn1 = tk.Button(root, text = "showinfo", command = btn1_clicked)
btn1.pack(fill = tk.X)
root.mainloop()
```



7.2.10 消息窗口（消息框）

Warning消息窗口

```
import tkinter as tk
from tkinter import messagebox as msgbox
def btn2_clicked():
    msgbox.showwarning("Warning", "Showwarning test.")
root = tk.Tk()
root.title("MsgBox Test")
btn2 = tk.Button(root, text = "showwarning", command = btn2_clicked)
btn2.pack(fill = tk.X)
root.mainloop()
```





7.2.10 消息窗口（消息框）

Error消息窗口

```
import tkinter as tk
from tkinter import messagebox as msgbox
def btn3_clicked():
    msgbox.showerror("Error", "Showerror test.")
root = tk.Tk()
root.title("MsgBox Test")
btn3 = tk.Button(root, text = "showerror", command = btn3_clicked)
btn3.pack(fill = tk.X)
root.mainloop()
```

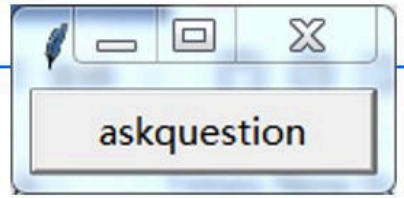


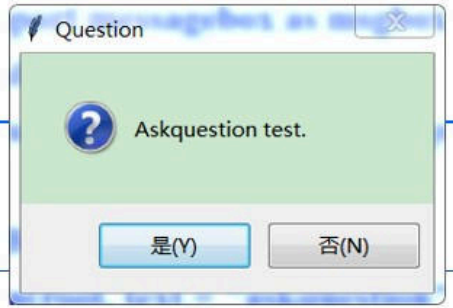


7.2.10 消息窗口（消息框）

Question消息窗口

```
import tkinter as tk
from tkinter import messagebox as msgbox
def btn4_clicked():
    result = msgbox.askquestion("Question", "Askquestion test.")
    print(result)   #点击【是】result = 'yes'，点击【否】result = 'no'
root = tk.Tk()
root.title("MsgBox Test")
btn4 = tk.Button(root, text = "askquestion", command = btn4_clicked)
btn4.pack(fill = tk.X)
root.mainloop()
```

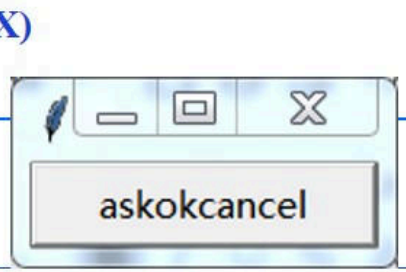


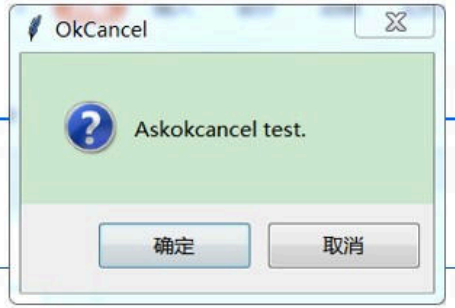


7.2.10 消息窗口（消息框）

OkCancel消息窗口

```
import tkinter as tk
from tkinter import messagebox as msgbox
def btn5_clicked():
    result = msgbox.askokcancel("OkCancel", "Askokcancel test.")
    print(result)    #result = True/False
root = tk.Tk()
root.title("MsgBox Test")
btn5 = tk.Button(root, text = "askokcancel", command = btn5_clicked)
btn5.pack(fill = tk.X)
root.mainloop()
```

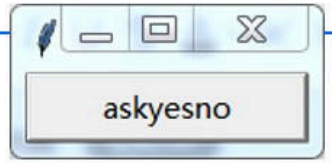


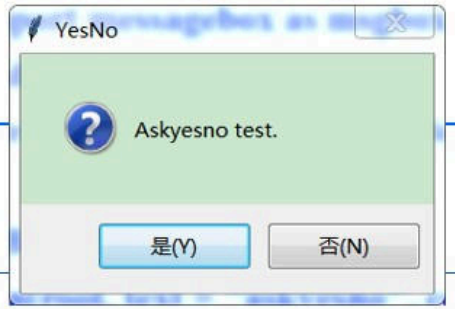


7.2.10 消息窗口（消息框）

YesNo消息窗口

```
import tkinter as tk
from tkinter import messagebox as msgbox
def btn6_clicked():
    result = msgbox.askyesno("YesNo", "Askyesno test.")
    print(result)                #result = True/False
root = tk.Tk()
root.title("MsgBox Test")
btn6 = tk.Button(root, text = "askyesno", command = btn6_clicked)
btn6.pack(fill = tk.X)
root.mainloop()
```

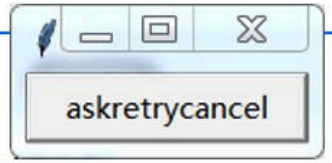


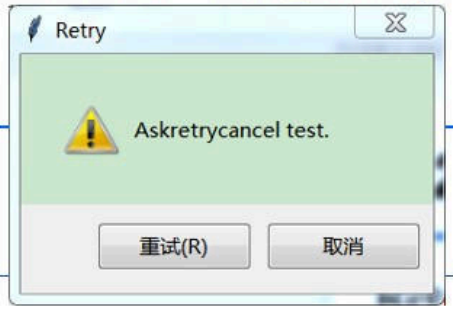


7.2.10 消息窗口（消息框）

Retry消息窗口

```
import tkinter as tk
from tkinter import messagebox as msgbox
def btn7_clicked():
    m = msgbox.askretrycancel("Retry", "Askretrycancel test.")
    print(m)        #点击重试返回'True'，点击取消返回'False'
root = tk.Tk()
root.title("MsgBox Test")
btn7 = tk.Button(root, text = "askretrycancel", command = btn7_clicked)
btn7.pack(fill = tk.X)
root.mainloop()
```





7.2.11 Frame框架组件

- Frame组件是框架组件，可以分组组织其他子组件，负责安排其他组件的位置。
- Frame组件在屏幕上显示为一个矩形区域，作为显示其他组件的容器。



7.2.11 Frame框架组件

1. 创建和显示Frame对象

- 创建Frame对象的方法：

  Frame对象 = Frame(窗口对象, height = 高度, width = 宽度, bg = 背景色, ...)

- 显示Frame对象的方法：　　**Frame对象.pack()**

2. 向Frame组件中添加子组件

- 在创建子组件的时候，指定它的容器为Frame组件即可，例如：

  Lable(Frame对象, text = 'Hello').pack()

3. 带有标题的Frame组件

- LableFrame组件是有标题的Frame组件，例如：

  LableFrame(窗口对象, height = 高度, width = 宽度, text = 标题).pack()

7.2.11 Frame框架组件

实例

```
import tkinter as tk
root = tk.Tk()                                  #创建窗口对象
root.title("使用Frame组件的例子")                #设置窗口标题
f1 = tk.Frame(root)                             #创建第1个Frame组件
f1.pack()
f2 = tk.Frame(root)                             #创建第2个Frame组件
f2.pack()
f3 = tk.LabelFrame(root,text = '第3个Frame')     #创建有标签的Frame框架
f3.pack( side = tk.BOTTOM )                     #放置在窗口底部
```


雨课堂 Rain Classroom

7.2.11 Frame框架组件

实例

```
redbutton = tk.Button(f1, text="Red", fg="red")
redbutton.pack( side = tk.LEFT)
brownbutton = tk.Button(f1, text="Brown", fg="brown")
brownbutton.pack( side = tk.LEFT )
bluebutton = tk.Button(f1, text="Blue", fg="blue")
bluebutton.pack( side = tk.LEFT )
blackbutton = tk.Button(f2, text="Black", fg="black")
blackbutton.pack()
greenbutton = tk.Button(f3, text="Green", fg="Green")
greenbutton.pack()
root.mainloop()
```



7.2.11 Frame框架组件

4. 刷新Frame对象

- 用Python做GUI图形界面，可以使用 **after** 方法每隔几秒刷新GUI图形界面，例如如下代码实现移动电子广告效果，使用after不断移动label。

```
import tkinter as tk
colors = ('red','orange','yellow','green','blue','purple')
root = tk.Tk()                                    #创建窗口对象
f = tk.Frame(root, height = 100, width = 200)
color = 0
f['bg'] = colors[color]
lab1 = tk.Label(f, text = r'人生苦短，我用Python')
x = 0
```


雨课堂 Rain Classroom

7.2.11 Frame框架组件

4. 刷新Frame对象

- 用Python做GUI图形界面，可以使用 **after** 方法每隔几秒刷新GUI图形界面，例如如下代码实现移动电子广告效果，使用after不断移动label。

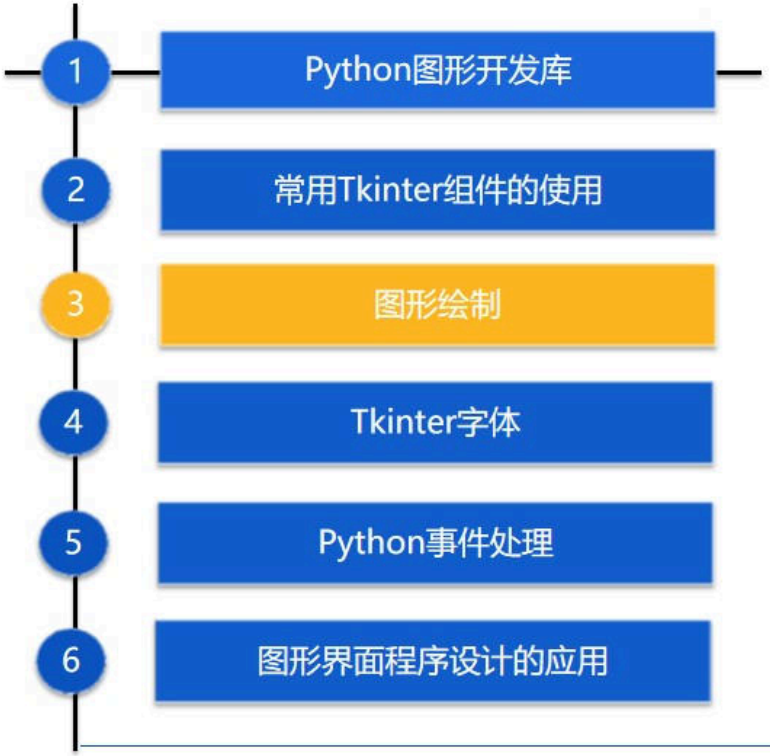
```
def foo():
    global x
    global color
    x = x+10
    if x>200:
        x = 0
    f['bg'] = colors[color]
    color = (color+1)%(len(colors))
    lab1.place(x = x, y = 30)
    f.after(500, foo)          #嵌套调用
```

```
f.pack()
foo()
root.mainloop()
```



目录

第7章 Tkinter图形界面设计



雨课堂 Rain Classroom

7.3 图形绘制

7.3.1 Canvas画布组件

- Canvas (画布)是一个长方形的区域，用于图形绘制或复杂的图形界面布局。可以在画布上绘制图形、文字，放置各种组件和框架。
- 创建Canvas对象的语法格式：

  Canvas对象 = Canvas(窗口对象，选项，.....)

- 显示Canvas对象的方法：

  Canvas对象.pack()

```
import tkinter as tk
root = tk.Tk()
cv = tk.Canvas(root, bg = 'white', width =300, height = 120)
cv.create_line(10, 10, 100, 80, width=2, dash=7)
cv.pack()
root.mainloop()
```


雨课堂 Rain Classroom

7.3 图形绘制

7.3.2 Canvas上的图形对象

- Canvas画布上可以绘制各种图形对象。通过调用如下绘制函数实现：
  - create_arc( ) 绘制圆弧。
  - create_line( ) 绘制直线。
  - create_bitmap( ) 绘制位图。
  - create_image( ) 绘制位图图像。
  - create_oval( ) 绘制椭圆。
  - create_polygon( ) 绘制多边形。
  - create_window( ) 绘制子窗口。
  - create_text( ) 创建一个文字对象

7.3 图形绘制

7.3.2 Canvas上的图形对象

- Canvas上每个绘制对象都会返回一个对象ID，例如：

```
id1 = cv.create_line(10, 10, 100, 80, width = 2, dash = 7 )          #绘制直线
```

- 移动指定图形对象：

```
Canvas对象.move(图形对象ID，x坐标偏移量，y坐标偏移量)
```

- 删除指定的图形对象

```
Canvas对象.delete(图形对象ID)
```

- 缩放指定的图形对象

```
Canvas对象.scale(图形对象ID，x轴偏移量，y轴偏移量，
                 x轴缩放比例，y轴缩放比例)
```

7.3 图形绘制

7.3.2 Canvas上的图形对象

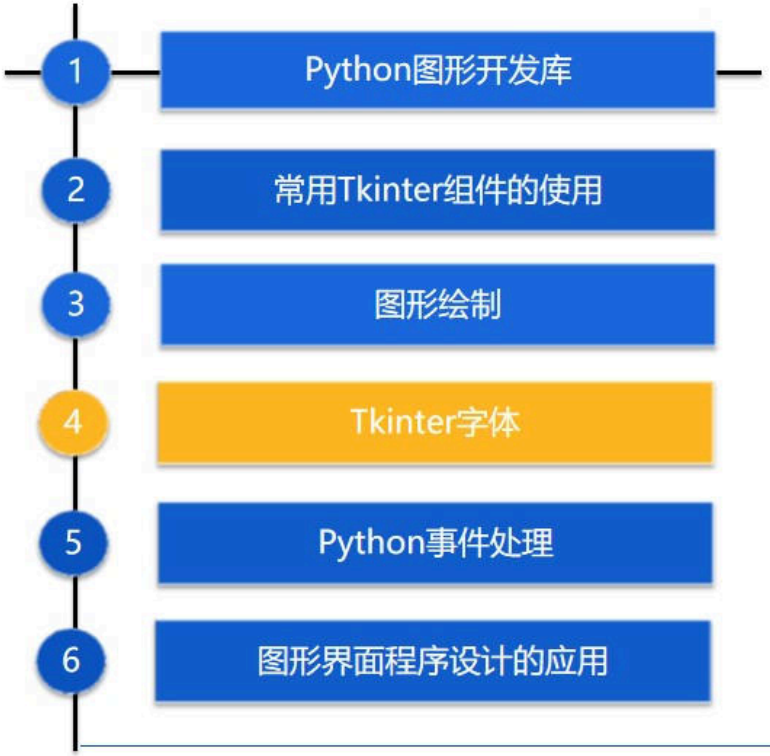
- 创建图形对象时可以使用属性tags设置图形的标签，拥有相同标签的图形可以设置共同的属性，如边框颜色等

```
import tkinter as tk
root = tk.Tk()
# 创建一个Canvas，设置其背景色为白色，
cv = tk.Canvas(root, bg = 'white', width = 200, height = 200)
#使用tags给第一个矩形指定3个tag
cv.create_rectangle(10,10,110,110, tags = ('r1','r2','r3'))
cv.pack()
cv.create_rectangle(20,20,80,80, tags = 'r3')  #使用tags给第2个矩形指定1个tag
for item in cv.find_withtag('r3'):  # 将所有与tag('r3')绑定的item边框颜色设置为蓝色
    cv.itemconfig(item,outline = 'blue')
root.mainloop()
```


雨课堂 Rain Classroom

目录

第7章 Tkinter图形界面设计



雨课堂 Rain Classroom

7.4 Tkinter字体

7.4.1 通过元组表示字体

- 通过组件的font属性，可以设置其显示文本的字体。
- 设置组件字体前首先要能表示一个字体，字体可以通过3个元素的元组来表示：（font family, size, modifiers） 其中，
  - font family是字体名；
  - size为字体大小单位为point；
  - modifiers为包含粗体、斜体、下划线的样式修饰符。




```
import tkinter as tk
root = tk.Tk()
for ft in ('Arial', ('Courier New',19,'italic'), ('Comic Sans MS',), 'Fixdsys', ('MS Sans Serif',), ('MS Serif',), 'Symbol','System', ('Times New Roman',), 'Verdana'):
    tk.Label(root, text = 'hello sticky', font = ft ).grid()
root.mainloop()
```

7.4 Tkinter字体

7.4.2 通过Font对象表示字体

- 也可以使用tkFont.Font来创建字体。格式如下：

```
ft = tkFont.Font(family = '字体名',size ,weight ,slant, underline, overstrike)
```

- 其中：
  - size为字体大小；
  - weight为字形——'bold'或'normal'；
  - slant='italic'或'normal'，'italic'为斜体；
  - underline=1或0，1为下划线；
  - overstrike=1或0，1为删除线。
- 例如：

```
ft = Font(family="Helvetica",size=36,weight="bold")
```


雨课堂 Rain Classroom

7.4 Tkinter字体

7.4.2 通过Font对象表示字体

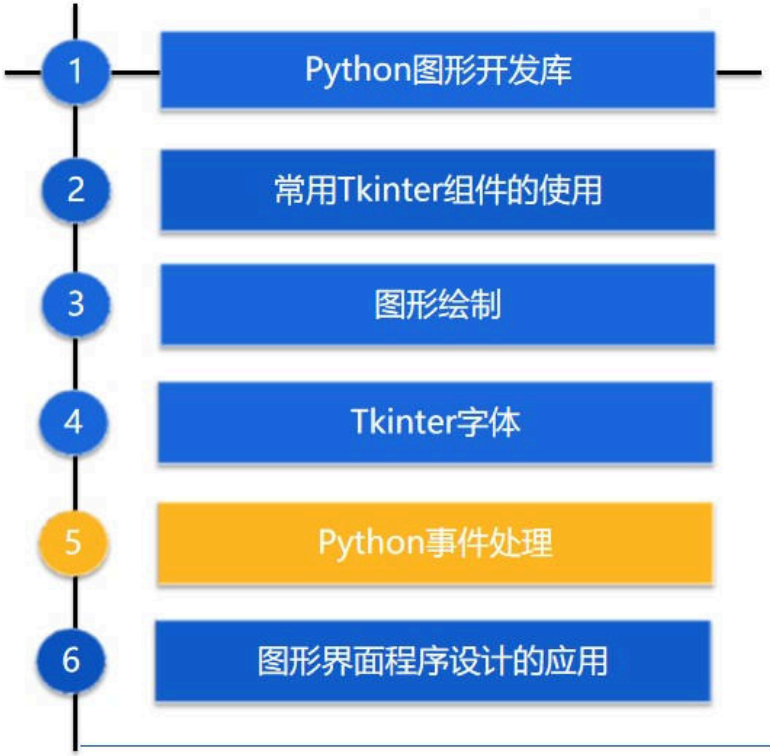
```
import tkinter as tk
import tkinter.font as ft                    # 引入字体模块
root = tk.Tk()

# 指定字体名称、大小、样式
ft = ft.Font(family = 'Fixdsys',size = 20,weight ='bold')
tk.Label(root, text = 'hello sticky', font = ft ).grid()        # 创建一个Label
root.mainloop()
```



目录

第7章 Tkinter图形界面设计



雨课堂 Rain Classroom

7.5 Python事件处理

- 所谓事件（event）就是程序上发生的事。例如用户敲击键盘上某一个键或是单击、移动鼠标。而对于这些事件，程序需要做出反应。
- Tkinter提供的组件通常都有自己可以识别的事件。例如当按钮被单击时执行特定操作或是当一个输入栏成为焦点，而你又敲击了键盘上的某些按键，你所输入的内容就会显示在输入栏内。
- 程序可以使用事件处理函数来指定当触发某个事件时所做的反应（操作）。

雨课堂 Rain Classroom

7.5.1 事件类型

事件类型的通用格式

- <[modifier-]…type[-detail]>
  - 事件类型必须放置于尖括号<>内。
  - modifier用于组合键定义，例如Control、Alt
  - type描述了类型，例如键盘按键、鼠标单击。
  - detail用于明确定义是哪一个键或按钮的事件，例如1表示鼠标左键、2表示鼠标中键、3表示鼠标右键。
- **Python中事件主要有：**
  - **键盘事件**
  - **鼠标事件**
  - **窗体事件**

雨课堂 Rain Classroom

7.5.1 事件类型

鼠标事件

- **鼠标单击事件**

<Button-1>	<Button-2>	<Button-3>	<Button-4>	<Button-5>
单击鼠标左键	单击鼠标中键	单击鼠标右键	向上滚动滑轮	向下滚动滑轮

- **鼠标双击事件**

<Double-Button-1>	<Double-Button-2>	<Double-Button-3>
鼠标左键双击	鼠标中键双击	鼠标右键双击

- **鼠标释放事件**

<ButtonRelease-1>	<ButtonRelease-2>	<ButtonRelease-3>
鼠标左键释放	鼠标中键释放	鼠标右键释放

雨课堂 Rain Classroom

7.5.1 事件类型

鼠标事件（续）

- **鼠标按下并移动事件（即拖动）**

<B1-Motion>	<B2-Motion>	<B3-Motion>
鼠标左键拖动	鼠标中键拖动	鼠标右键拖动

- **鼠标其他操作事件**

<Enter>	<FocusIn>	<Leave>	<FocusOut>
鼠标进入控件（放到控件上面）	控件获得焦点	鼠标移出控件	控件失去焦点

7.5.1 事件类型

键盘事件

- **键盘按下事件**
  - **<Key>**：键盘按下，事件event中的keycode, char都可以获取按下的键值
  - **<KeyPress>**：按下键盘某键时触发，可以在detail部分指定是哪个键
  - **<KeyRelease>**：释放键盘某键时触发，可以在detail部分指定是哪个键
  - **<Return>**:键位绑定，回车键，其它还有<BackSpace>, <Escape>, <Left>, <Up>, <Right>, <Down>等等

- **控件属性改变事件**
  - **<Configure>：控件大小改变，新的控件大小会存储在事件event对象中的width 和 height 属性传递，部分平台上该事件也代表控件位置改变。**

雨课堂 Rain Classroom

7.5.1 事件类型

Python中事件主要有：键盘事件，鼠标事件，窗体事件

- 事件类型的通用格式：<[modifier-]…type[-detail]>
- **键盘事件**

名称	描述
KeyPress	按下键盘某键时触发，可以在detail部分指定是哪个键
KeyRelease	释放键盘某键时触发，可以在detail部分指定是哪个键

雨课堂 Rain Classroom

7.5.1 事件类型

Python中事件主要有：键盘事件，鼠标事件，窗体事件

- 事件类型的通用格式：<[modifier-]…type[-detail]>
- **鼠标事件**

名称	描述
ButtonPress或Button	按下鼠标某键时触发，可以在detail部分指定是哪个键
ButtonRelease	释放鼠标某键时触发，可以在detail部分指定是哪个键
Motion	点中组件的同时拖曳组件移动时触发
Enter	当鼠标指针移进某个组件时触发
Leave	当鼠标指针移出某个组件时触发
MouseWheel	当鼠标滚轮滚动时触发

雨课堂 Rain Classroom

7.5.1 事件类型

Python中事件主要有：键盘事件，鼠标事件，窗体事件

- **窗体事件**

名称	描述
visibility	当组件变为可视状态时触发
Unmap	当组件由显示状态变为隐藏状态时触发
Map	当组件由隐藏状态变为显示状态时触发
Expose	当组件由原本被其他组件遮盖的状态中暴露出来时触发
FocusIn	组件获得焦点时触发
FocusOut	组件失去焦点时触发
Configure	当改变组件大小时触发。例如拖曳窗体边缘
Property	当窗体的属性呗删除或改变时触发，属于Tk的核心事件
Destroy	当组件被销毁时触发
Activate	与组件中的状态选项有关，表示由不可用转为可用时触发
Deactivate	与组件中的状态选项有关，表示由可用转为不可用时触发

雨课堂 Rain Classroom

7.5.2 事件绑定

1．创建组件对象时指定

- 程序建立一个处理某一事件的事件处理函数，称之为**绑定。**
- 创建组件对象实例时，可通过其命名参数command指定事件处理函数。例

```
import tkinter as tk
from tkinter import messagebox
root = tk.Tk()

def callback():    #定义一个事件处理函数
    messagebox.showinfo("Python command","人生苦短、我用Python")

Bu1=tk.Button(root, text="设置事件调用命令",command=callback)
Bu1.pack()
root.mainloop()
```


雨课堂 Rain Classroom

7.5.2 事件绑定

2．实例绑定（最常用事件绑定方式）

- 调用组件对象实例方法bind可为指定组件实例绑定事件。

  组件对象实例名.bind("<事件类型>", 事件处理函数)

- 例如假设声明了一个名为canvas的Canvas组件对象，想在canvas上按下鼠标左键时画上一条线，可以这样实现：

  canvas.bind("<Button-1>", drawline)

  - 其中，第一个参数是事件描述符，第二个参数是事件处理函数，这里只是函数声明，不需要后面的圆括号，Tkinter会把此函数填入参数然后调用运行。

7.5.2 事件绑定

2．实例绑定

- 例如：

```
import tkinter as tk
def drawline(event):
    cv.create_line(10,10,200,100,width =3,dash=7)
root = tk.Tk()
cv = tk.Canvas(root, bg = 'white', width = 200, height = 120)
cv.bind("<Button-1>",drawline)
cv.create_rectangle(20,20,110,110,outline='red',stipple='gray12',fill='green')
cv.pack()
root.mainloop()
```


雨课堂 Rain Classroom

7.5.2 事件绑定

3．类绑定

- 将事件与一组件类绑定。调用任意组件实例的.bind_class()函数为特定组件类绑定事件。

  组件实例名.bind_class ("组件类","<事件类型>", 事件处理函数)

例如：

```
import tkinter as tk
def drawline(event):
    cv.create_line(10,10,200,100,width =3,dash=7)
root = tk.Tk()
cv = tk.Canvas(root, bg = 'white', width = 200, height = 120)
cv.create_rectangle(20,20,110,110,outline='red',stipple='gray12',fill='green')
cv.bind_class("Canvas","<Button-1>",drawline)
cv.pack()
root.mainloop()
```

25

7.5.2 事件绑定

4. 程序界面绑定

- 当无论在哪一组件实例上触发某一事件，程序都作出相应的处理。
- 例如，可以将PrintScreen键与程序中的所有组件对象绑定，这样整个程序界面就能处理打印屏幕的事件了。调用任意组件实例的.bind_all()函数为程序界面绑定事件。

  组件实例名.bind_ all ("<事件类型>", 事件处理函数)
- 例如可以这样实现打印屏幕：

  widget.bind_all("<Key-Print>",printScreen)。

雨课堂 Rain Classroom

7.5.2 事件绑定

4．程序界面绑定

- 调用任意组件实例的.bind_all()函数为程序界面绑定事件。

  组件实例名.bind_ all ("<事件类型>", 事件处理函数)

- 例如可以这样实现打印屏幕：

  widget.bind_all("<Key-Print>",printScreen)。

```
import tkinter as tk
def drawline(event):
    cv.create_line(10,10,200,100,width =3,dash=7)
root = tk.Tk()
cv = tk.Canvas(root, bg = 'white', width = 200, height = 120)
cv.create_rectangle(20,20,110,110,outline='red',stipple='gray12',fill='green')
cv.bind_all("<Button-1>",drawline)
cv.pack()
root.mainloop()
```


雨课堂 Rain Classroom

7.5.2 事件绑定

5．标识绑定

- 在Canvas画布中绘制各种图形，将图形与事件绑定可以使用标识绑定tag_bind()函数。预先为图形定义标识tag后，通过标识tag来绑定事件。例如：**cv.tag_bind('r1','<Button-1>',printRect)**

```
import tkinter as tk
def drawline(event):
    cv.create_line(10,10,200,100,width =3,dash=7)
root = tk.Tk()
cv = tk.Canvas(root, bg = 'white', width = 200, height = 120)
cv.create_rectangle(20,20,110,110,outline='red',stipple='gray12',fill='green',
tags = 'rt')
cv.tag_bind("rt", "<Button-1>", drawline)
cv.pack()
root.mainloop()
```

7.5.3 事件处理函数

1. 定义事件处理函数

- 事件处理函数往往带有一个event参数。触发事件调用事件处理函数时，将传递Event对象实例。

```
def callback(event):        #事件处理函数
    showinfo("Python command","人生苦短、我用Python")
```


雨课堂 Rain Classroom

7.5.3 事件处理函数

2. Event事件处理参数的属性

- Event对象实例可以获取各种相关的参数。Event事件对象主要参数属性如

名称	描述
.x, .y	鼠标相对于组件对象左上角的坐标
.x_root, .y_root	鼠标相对于屏幕对象左上角的坐标
.keysym	字符串命名按键
.keysym_num	数字代码命名按键
.keycode	键码，但是它不能反映事件前缀Alt, Control, Shift, Lock，并且不区分大小写按键
.time	时间
.type	事件类型
.widget	触发事件的对应组件
.char	字符

雨课堂 Rain Classroom

7.5.3 事件处理函数

2. Event事件处理参数的属性

- Event事件对象按键详细信息说明

.keysym	.keycode	.keysym_num	说明
Alt_L	64	65513	左侧Alt键
Alt_R	113	65514	右侧Alt键
BackSpace	22	65288	BackSpace键
Cancel	110	65387	Pause Break键
F1~F11	67~77	65470~65480	功能键F1~F11
Print	111	65377	打印屏幕键

7.5.3 事件处理函数

2. 定义事件处理函数——实例

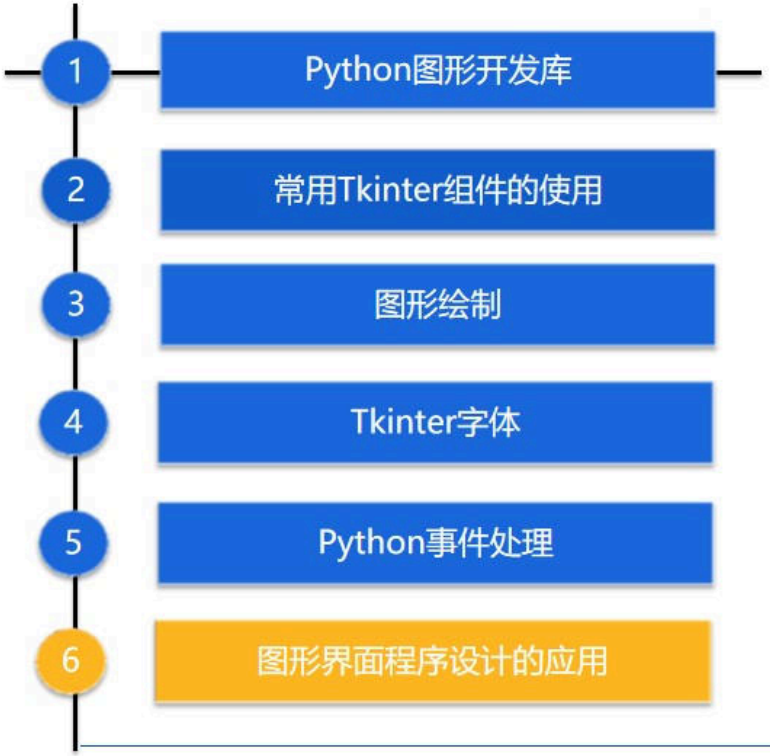
- 触发keyPress键盘事件的例子

```
import tkinter as tk                          #导入tkinter
def printkey(event):                          #定义的函数监听键盘事件
    print('你按下了:' + event.char)
root = tk.Tk()                                #实例化tk
entry = tk.Entry(root)                        #实例化一个单行输入框

#给输入框绑定按键监听事件<KeyPress>以监听任何按键
entry.bind('<KeyPress>', printkey)
entry.pack()
root.mainloop()
```

目录

第7章 Tkinter图形界面设计



雨课堂 Rain Classroom

7.6.1 开发猜数字游戏

使用tkinter开发猜数字游戏，运行效果如图所示。

- 游戏中电脑随机生成1024以内数字，玩家去猜，如果猜得数字过大过小都会提示，程序要统计玩家猜的次数。



7.6.1 开发猜数字游戏

使用tkinter开发猜数字游戏，运行效果如图所示。

```
import tkinter as tk
import sys
import random
import re

number = random.randint(0,1024)                    #玩家要猜的数字
running = True
num = 0                                            #猜的次数
nmaxn = 1024                                       #提示猜测范围的最大数
nminn = 0                                          #提示猜测范围的最小数

def eBtnClose(event):
    root.destroy()
```

7.6.1 开发猜数字游戏

使用tkinter开发猜数字游戏，运行效果如图所示。

```
def eBtnGuess(event):
    global nmaxn
    global nminn
    global num
    global running
    if running:
        val_a = int(entry_a.get())      #获取猜的数字输入并转换成数字
        if val_a == number:
            labelqval("恭喜答对了！ ")
            num+=1
            running = False
            numGuess()                  #显示猜的次数
        elif val_a < number:            #猜小了
            if val_a > nminn:
                nminn = val_a           #修改提示猜测范围的最小数
                num+=1
                labelqval("小了哦,请输入"+str(nminn)+"到"+str(nmaxn)+"之间任意整数： ")
        else:
            if val_a < nmaxn:
                nmaxn = val_a           #修改提示猜测范围的最大数
                num+=1
                labelqval("大了哦,请输入"+str(nminn)+"到"+str(nmaxn)+"之间任意整数： ")
    else:
        labelqval('你已经答对啦...')
```

7.6.1 开发猜数字游戏

使用tkinter开发猜数字游戏，运行效果如图所示。

```
#显示猜的次数
def numGuess():
    if num == 1:
        labelqval('我靠！一次答对！')
    elif num < 10:
        labelqval('==十次以内就答对了牛。。。尝试次数：'+str(num))
    else:
        labelqval('好吧，您都试了超过10次了。。。。尝试次数：'+str(num))

def labelqval(vText):
    label_val_q.config(label_val_q,text=vText)        #修改提示标签文字
```


雨课堂 Rain Classroom

7.6.1 开发猜数字游戏

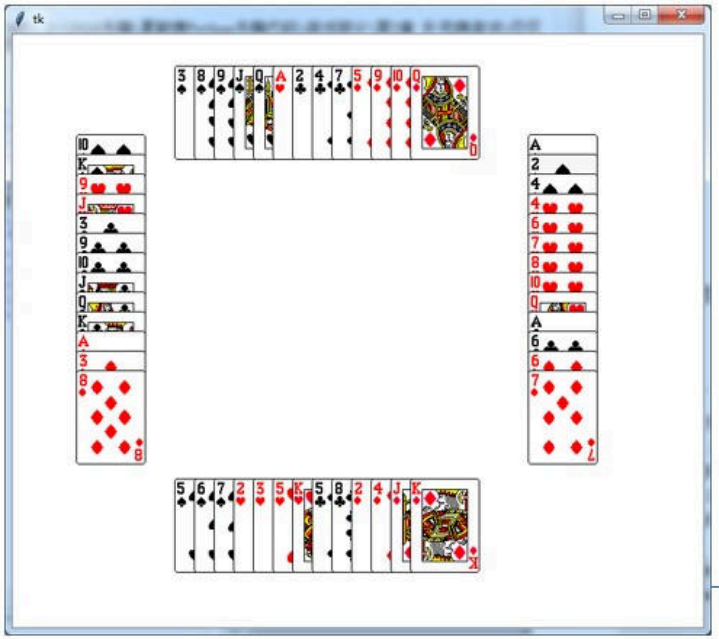
使用tkinter开发猜数字游戏，运行效果如图所示。

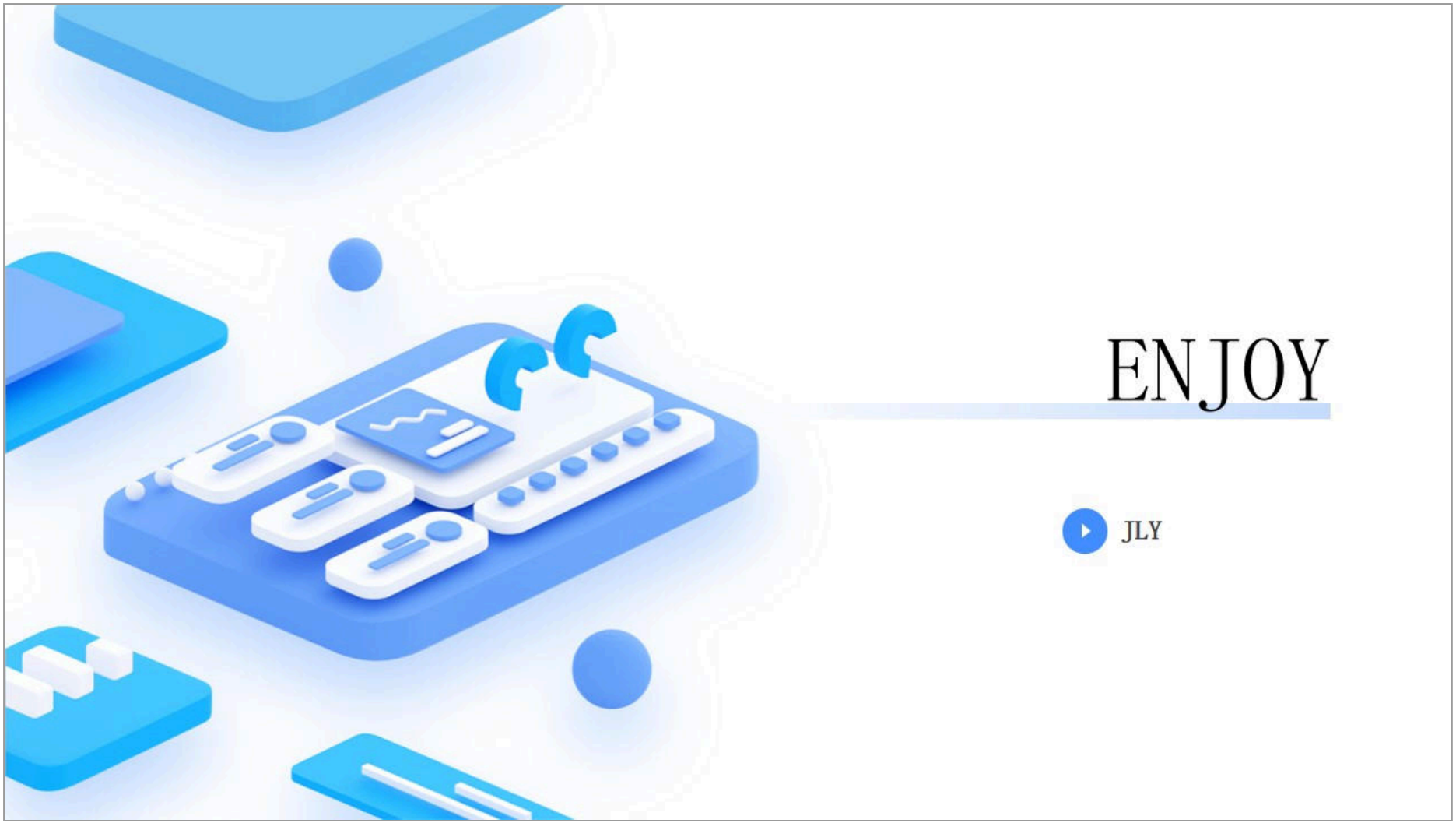
```
root = tk.Tk(className="猜数字游戏")
root.geometry("400x90+200+200")
label_val_q = tk.Label(root,width="80")          #提示标签
label_val_q.pack(side = "top")
entry_a = tk.Entry(root,width="40")              #单行输入文本框
btnGuess = tk.Button(root,text="猜")             #猜按钮
entry_a.pack(side = "left")
entry_a.bind('<Return>',eBtnGuess)               #绑定事件
btnGuess.bind('<Button-1>',eBtnGuess)            #猜按钮
btnGuess.pack(side = "left")
btnClose = tk.Button(root,text="关闭")           #关闭按钮
btnClose.bind('<Button-1>',eBtnClose)
btnClose.pack(side="left")
labelqval("请输入0到1024之间任意整数：")
entry_a.focus_set()
print(number)
root.mainloop()
```

7.6.2 扑克牌发牌程序窗体图形版

游戏初步——扑克牌发牌程序窗体图形版

- 4名牌手打牌，电脑随机将52张牌（不含大小鬼）发给4名打牌，在屏幕上显示每位牌手的牌。程序的运行效果如图所示。




ENJOY
JLY